# HashiCorp Vault — Lab Guide, Day 3

Oct 16, 2026 · @Vivek Arora

## How to use this lab guide

This guide holds every hands-on step for Day 3. The Day 3 study guide explains the concepts and points here with "Do Lab 3.x"; each lab names the module it practises. It continues the Day 1–2 Lab Guide: same machine, same conventions, same reset style. Labs 1.x and 2.x are not prerequisites to run these labs, but Lab 3.10 assumes you have done Labs 3.1, 3.2 and 3.4.

### Lab map

| Lab | Title | Time | Study guide module | Environment |
|---|---|---|---|---|
| 3.1 | PKI: two-tier CA, issue, verify, revoke | 35 min | Day 3 · Module 1 | Dev server |
| 3.2 | Transit: encrypt, rotate, rewrap, data keys | 25 min | Day 3 · Module 2 | Dev server |
| 3.3 | Transit auto-unseal and recovery keys | 25 min | Day 3 · Module 3 | Dev server + seal node |
| 3.4 | Vault Agent: templates, pkiCert, exec, API proxy | 35 min | Day 3 · Module 4 | Dev server + Lab 3.1 |
| 3.5 | Three-node Raft: failover and snapshots | 35 min | Day 3 · Module 5 | Raft cluster |
| 3.6 | Telemetry, Prometheus and audit devices | 25 min | Day 3 · Module 6 | Dev server + Docker |
| 3.7 | Break-fix troubleshooting | 30 min | Day 3 · Module 7 | Dev server + dev-TLS server |
| 3.8 | Terraform and Ansible automation | 40 min | Day 3 · Module 8 | Raft cluster (Lab 3.5) |
| 3.9 | Vault HA on Kubernetes with Helm and VSO | 45 min | Day 3 · Module 9 | kind cluster |
| 3.10 | Capstone: PKI + Transit + Vault Agent | 45 min | Day 3 · Module 10 | Dev server + nginx |

### Lab environments

| Name | How to start it | Ports | Used by |
|---|---|---|---|
| Dev server | `vault server -dev -dev-root-token-id=root` in terminal 1, then `export VAULT_ADDR=http://127.0.0.1:8200 VAULT_TOKEN=root` | 8200 | 3.1–3.4, 3.6, 3.7, 3.10 |

| Name | How to start it | Ports | Used by |
|---|---|---|---|
| Seal node | Built in Lab 3.3 under `~/day3/seal` | 8210/8211 | 3.3 |
| Raft cluster | `raft-cluster.sh` from Lab 3.5, data in `~/day3/raft` | 8210–8231 | 3.5, 3.8 |
| Dev-TLS server | `vault server -dev -dev-tls` in Lab 3.7 | 8300 | 3.7 |
| kind cluster | Built in Lab 3.9 | — | 3.9 |

The seal node and the Raft cluster both use ports 8210/8211, so finish and clean up Lab 3.3 before Lab 3.5. The dev server on 8200 can keep running beside the Raft cluster. As on Days 1–2, a dev server forgets everything when it stops: run Labs 3.1, 3.2 and 3.4 in one dev-server session, because 3.4 uses the PKI mounts from 3.1.

## One-time machine setup (additions for Day 3)

The Day 1–2 setup (Vault from the HashiCorp apt repository, Docker, jq, kubectl, helm, kind) is assumed. Add:

```
# Tools used in Day 3
$ sudo apt update
$ sudo apt install -y openssl sqlite3 nginx python3-requests terraform pipx
$ sudo systemctl disable --now nginx            # Lab 3.10 runs its own nginx as your user
$ pipx install --include-deps ansible && pipx inject ansible hvac
$ ansible-galaxy collection install community.hashi_vault
$ mkdir -p ~/day3
```

`terraform` comes from the same HashiCorp apt repository you added for Vault in Lab 1.2.

## Conventions

- `$` lines are typed at the shell; `/ $` lines are typed inside a container or pod.
- File blocks start with a comment naming the file (for example `# agent.hcl`); create the file with exactly that content.
- Every **Expected output** image shows what a correct run prints, from Vault 2.1.x. Tokens, keys, serial numbers, IDs, timestamps and version numbers will differ on your machine; the highlighted (amber) lines are the ones to check.
- ✓ **Checkpoint** lines tell you what must be true before moving on.
- Root tokens, `-key-shares=1` and `tls_disable` are lab shortcuts. Each lab says what production does instead; the study guide has the production configuration.
- All passwords, tokens, keys and card numbers in this guide are lab-only test values.

# Lab 3.1 — PKI: two-tier CA, issue, verify, revoke

**Objective:** build a root CA and an intermediate CA in Vault, issue a short-lived server certificate from a role, verify the chain with OpenSSL, and revoke it. **Time:** 35 min. **Study guide:** Day 3, Module 1. **Environment:** dev server (keep it running for Labs 3.2 and 3.4).

## Steps

**Step 1 — Start a dev server and a working folder.** In terminal 1:

```
$ vault server -dev -dev-root-token-id=root
```

In terminal 2:

```
$ export VAULT_ADDR=http://127.0.0.1:8200 VAULT_TOKEN=root
$ mkdir -p ~/day3/pki && cd ~/day3/pki
```

**Step 2 — Create the root CA.** The root lives in its own mount with a 10-year maximum lifetime. `issuer_name` gives it a stable name to refer to later.

```
$ vault secrets enable pki
$ vault secrets tune -max-lease-ttl=87600h pki
$ vault write pki/root/generate/internal common_name="StackMaster Root CA" issuer_name=root-
2026 ttl=87600h | cut -c1-90
$ vault write pki/config/urls issuing_certificates="$VAULT_ADDR/v1/pki/ca"
crl_distribution_points="$VAULT_ADDR/v1/pki/crl"
```

Lab 3.1 · Step 2 · Create the root CA — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault secrets enable pki
Success! Enabled the pki secrets engine at: pki/
trainee@vault-lab:~$ vault secrets tune -max-lease-ttl=87600h pki
Success! Tuned the secrets engine at: pki/
trainee@vault-lab:~$ vault write pki/root/generate/internal common_name="StackMaster Root CA"
                     issuer_name=root-2026 ttl=87600h | cut -c1-90
Key             Value
---             -----
certificate     -----BEGIN CERTIFICATE-----
MIIDTjCCAjagAwIBAgIUG7cB1Hq0wQ2oN0a8m3hJxW5r4XAwDQYJKoZIhvcNAQELBQAwHjEcMBoGA1UE...
expiration      2036-10-13T04:58:10Z
issuer_id       5a3c1e0f-7d2b-4c8e-9a61-2f0b7d93c4ab
issuer_name     root-2026
issuing_ca        -----BEGIN CERTIFICATE-----
key_id          c81f2d44-0b1a-4b5e-8e77-63d9a0f1b2c5
key_name        n/a
serial_number   1b:b7:01:d4:7a:b4:c1:0d:a8:37:46:bc:9b:78:49:c5:6e:6b:e1:70
trainee@vault-lab:~$ vault write pki/config/urls issuing_certificates="$VAULT_ADDR/v1/pki/ca"
                     crl_distribution_points="$VAULT_ADDR/v1/pki/crl"
Key                       Value
---                       -----
crl_distribution_points   [http://127.0.0.1:8200/v1/pki/crl]
issuing_certificates      [http://127.0.0.1:8200/v1/pki/ca]
```

*Expected output — root CA generated with issuer root-2026; issuing and CRL URLs set*

`internal` means the private key is generated inside Vault and can never be read out. (`exported` would return it once — avoid it for a CA.)

**Step 3 — Create the intermediate CA and have the root sign it.**

```
$ vault secrets enable -path=pki_int pki && vault secrets tune -max-lease-ttl=43800h pki_int
$ vault write -field=csr pki_int/intermediate/generate/internal common_name="StackMaster
Intermediate CA" issuer_name=int-2026 > pki_int.csr
$ vault write -field=certificate pki/root/sign-intermediate issuer_ref=root-2026
csr=@pki_int.csr format=pem_bundle ttl=43800h > int.pem
$ vault write pki_int/intermediate/set-signed certificate=@int.pem
$ vault write pki_int/config/urls issuing_certificates="$VAULT_ADDR/v1/pki_int/ca"
crl_distribution_points="$VAULT_ADDR/v1/pki_int/crl"
```

**Step 4 — Create a role.** The role is the policy for certificates: which names, how long, which key type.

```
$ vault write pki_int/roles/web-server allowed_domains=stackmaster.lab allow_subdomains=true
max_ttl=720h
```

Lab 3.1 · Steps 3-4 · Intermediate CA and role — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault secrets enable -path=pki_int pki && vault secrets tune
                     -max-lease-ttl=43800h pki_int
Success! Enabled the pki secrets engine at: pki_int/
Success! Tuned the secrets engine at: pki_int/
trainee@vault-lab:~$ vault write -field=csr pki_int/intermediate/generate/internal
                     common_name="StackMaster Intermediate CA" issuer_name=int-2026 >
                     pki_int.csr
trainee@vault-lab:~$ vault write -field=certificate pki/root/sign-intermediate
                     issuer_ref=root-2026 csr=@pki_int.csr format=pem_bundle ttl=43800h >
                     int.pem
trainee@vault-lab:~$ vault write pki_int/intermediate/set-signed certificate=@int.pem
Key                 Value
---                 -----
existing_issuers    <nil>
existing_keys       <nil>
imported_issuers    [9e2d7b51-3f0c-4a18-b6d4-0c7e1a5f8b23]
imported_keys       <nil>
mapping
    map[9e2d7b51-3f0c-4a18-b6d4-0c7e1a5f8b23:4f6a0c2e-8b1d-47e3-a9c5-d21b7e60f3a8]
trainee@vault-lab:~$ vault write pki_int/roles/web-server allowed_domains=stackmaster.lab
                     allow_subdomains=true max_ttl=720h
Key                                 Value
---                                 -----
allow_subdomains                    true
allowed_domains                     [stackmaster.lab]
key_type                            rsa
max_ttl                             720h
...
```

*Expected output — intermediate imported as a new issuer; role web-server created*

✓ **Checkpoint:** `imported_issuers` shows one ID, and `vault list pki_int/issuers` lists it.

**Step 5 — Issue a certificate.** `issue` makes Vault generate the key pair too; save the parts to files.

```
$ vault write -format=json pki_int/issue/web-server common_name=www.stackmaster.lab ttl=24h >
www.json
$ jq -r '.data.serial_number, .data.expiration, .data.private_key_type' www.json
```

```
$ jq -r .data.certificate www.json > www.crt
$ jq -r .data.private_key www.json > www.key && chmod 600 www.key
$ jq -r '.data.ca_chain[]' www.json > chain.pem
$ ls -l www.*
```

Lab 3.1 · Step 5 · Issue a certificate — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault write -format=json pki_int/issue/web-server
                       common_name=www.stackmaster.lab ttl=24h > www.json
trainee@vault-lab:~$ jq -r '.data.serial_number, .data.expiration, .data.private_key_type'
                       www.json
3c:9a:51:0e:77:d2:4b:18:a0:6f:c3:92:1d:e4:58:b7:20:0a:6c:91
1792213364
rsa
trainee@vault-lab:~$ jq -r .data.certificate www.json > www.crt
trainee@vault-lab:~$ jq -r .data.private_key www.json > www.key && chmod 600 www.key
trainee@vault-lab:~$ jq -r '.data.ca_chain[]' www.json > chain.pem
trainee@vault-lab:~$ ls -l www.*
-rw-rw-r-- 1 trainee trainee 1294 Oct 16 10:32 www.crt
-rw-rw-r-- 1 trainee trainee 4387 Oct 16 10:32 www.json
-rw------- 1 trainee trainee 1675 Oct 16 10:32 www.key
```

*Expected output — serial number, expiry and key type; certificate and key saved*

The private key is in the response only once — Vault does not store it. That is why the key file must be saved now, and why apps usually get certificates through Vault Agent (Lab 3.4).

**Step 6 — Inspect and verify the chain.**

```
$ openssl x509 -in www.crt -noout -subject -issuer -dates -ext subjectAltName
$ vault read -field=certificate pki/issuer/root-2026 > root_ca.crt
$ openssl verify -CAfile root_ca.crt -untrusted int.pem www.crt
```

Lab 3.1 · Step 6 · Inspect and verify the chain — EXPECTED OUTPUT

```
trainee@vault-lab:~$ openssl x509 -in www.crt -noout -subject -issuer -dates -ext subjectAltName
subject=CN = www.stackmaster.lab
issuer=CN = StackMaster Intermediate CA
notBefore=Oct 16 05:02:14 2026 GMT
notAfter=Oct 17 05:02:44 2026 GMT
X509v3 Subject Alternative Name:
    DNS:www.stackmaster.lab
trainee@vault-lab:~$ vault read -field=certificate pki/issuer/root-2026 > root_ca.crt
trainee@vault-lab:~$ openssl verify -CAfile root_ca.crt -untrusted int.pem www.crt
www.crt: OK
```

*Expected output — issued by the intermediate, valid for 24 hours, chain verifies OK*

✓ **Checkpoint:** the issuer is `StackMaster Intermediate CA`, `notAfter` is 24 hours after `notBefore`, and `openssl verify` prints `OK`.

**Step 7 — Revoke, read the CRL, and test the role limits.**

```
$ SERIAL=$(jq -r .data.serial_number www.json)
$ vault write pki_int/revoke serial_number=$SERIAL
$ curl -s $VAULT_ADDR/v1/pki_int/crl/pem | openssl crl -noout -text | grep -A1 "Serial
```

```
Number"
$ vault write pki_int/issue/web-server common_name=www.example.com
```

Lab 3.1 · Step 7 · Revoke, CRL and role limits — EXPECTED OUTPUT

```
trainee@vault-lab:~$ SERIAL=$(jq -r .data.serial_number www.json)
trainee@vault-lab:~$ vault write pki_int/revoke serial_number=$SERIAL
Key                        Value
---                        -----
revocation_time            1792127564
revocation_time_rfc3339    2026-10-16T05:12:44.183922Z
state                      revoked
trainee@vault-lab:~$ curl -s $VAULT_ADDR/v1/pki_int/crl/pem | openssl crl -noout -text | grep
                       -A1 "Serial Number"
    Serial Number: 3C9A510E77D24B18A06FC3921DE458B7200A6C91
        Revocation Date: Oct 16 05:12:44 2026 GMT
trainee@vault-lab:~$ vault write pki_int/issue/web-server common_name=www.example.com
Error writing data to pki_int/issue/web-server: Error making API request.

URL: PUT http://127.0.0.1:8200/v1/pki_int/issue/web-server
Code: 400. Errors:

* common name www.example.com not allowed by this role
```

*Expected output — certificate revoked and listed on the CRL; example.com refused by the role*

**Step 8 — Housekeeping.** Tidy removes expired and revoked entries from storage; the health check reviews the mount against HashiCorp's recommendations.

```
$ vault write pki_int/tidy tidy_cert_store=true tidy_revoked_certs=true safety_buffer=72h
$ vault write pki_int/config/auto-tidy enabled=true tidy_cert_store=true
tidy_revoked_certs=true interval_duration=12h
$ vault pki health-check pki_int
```

The health check prints a table per check ( `ok` , `informational` , `warning` ). On a lab mount expect warnings such as the root and intermediate living on the same Vault, or no ACME configured — read each message and compare with the study guide's production notes.

**Step 9 (optional) — Sign a CSR instead of issuing.** When the key must never leave the server:

```
$ openssl req -new -newkey rsa:2048 -nodes -keyout api.key -subj "/CN=api.stackmaster.lab" -
out api.csr
$ vault write -field=certificate pki_int/sign/web-server csr=@api.csr ttl=24h > api.crt
$ openssl x509 -in api.crt -noout -subject -issuer
```

## Clean up

Keep the dev server, the `pki` and `pki_int` mounts and the `web-server` role — Labs 3.2 and 3.4 build on this session. `~/day3/pki` holds `root_ca.crt` , which Lab 3.4 also uses.

# Lab 3.2 — Transit: encrypt, rotate, rewrap, data keys

**Objective:** use Vault as an encryption service: encrypt and decrypt a card number, rotate the key, rewrap old ciphertext, enforce a decryption floor, generate a data key, and give an app an encrypt-only policy. **Time:** 25 min. **Study guide:** Day 3, Module 2. **Environment:** dev server from Lab 3.1 ( `VAULT_TOKEN=root` ).

## Steps

**Step 1 — Enable Transit, create a key, encrypt.** Transit takes base64 input, so binary data works too.

```
$ vault secrets enable transit
$ vault write -f transit/keys/orders
$ vault write transit/encrypt/orders plaintext=$(echo -n "4111 1111 1111 1111" | base64)
```

Lab 3.2 · Step 1 · Create a key and encrypt — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault secrets enable transit
Success! Enabled the transit secrets engine at: transit/
trainee@vault-lab:~$ vault write -f transit/keys/orders
Key                       Value
---                       -----
allow_plaintext_backup    false
auto_rotate_period        0s
deletion_allowed          false
exportable                false
latest_version            1
min_decryption_version    1
name                      orders
type                      aes256-gcm96
trainee@vault-lab:~$ vault write transit/encrypt/orders plaintext=$(echo -n "4111 1111 1111
                         1111" | base64)
Key            Value
---            -----
ciphertext     vault:v1:8SDd3WHDOjf7mq69CyCqYjBXAiQQAVZRkFM13ok481zoCmHnSeDX9vyf7w==
key_version    1
```

*Expected output — aes256-gcm96 key at version 1; ciphertext starts with vault:v1:*

The prefix tells you the engine ( `vault` ) and the key version ( `v1` ). Store the whole string, prefix included.

**Step 2 — Decrypt, and encrypt the same value twice.** Paste your own ciphertext from Step 1.

```
$ CT1="vault:v1:...your ciphertext..."
$ vault write -field=plaintext transit/decrypt/orders ciphertext=$CT1 | base64 -d; echo
$ vault write transit/encrypt/orders plaintext=$(echo -n "4111 1111 1111 1111" | base64)
```

Lab 3.2 · Step 2 · Decrypt; a fresh nonce each time — EXPECTED OUTPUT

```
trainee@vault-lab:~$ CT1="vault:v1:8SDd3WHDOjf7mq69CyCqYjBXAiQQAVZRkFM13ok481zoCmHnSeDX9vyf7w=="
trainee@vault-lab:~$ vault write -field=plaintext transit/decrypt/orders ciphertext=$CT1 |
                       base64 -d; echo
4111 1111 1111 1111
trainee@vault-lab:~$ vault write transit/encrypt/orders plaintext=$(echo -n "4111 1111 1111
                       1111" | base64)
Key           Value
---           -----
ciphertext    vault:v1:Xq0kq3m4m3gW2bq+r1hTt0bQvKpE8l7mZ3d1sF2d6pTq9cYb0LwZ3A==
key_version   1
Same plaintext, different ciphertext: each call uses a fresh nonce.
```

*Expected output — decryption returns the card number; a second encryption gives different ciphertext*

✓ **Checkpoint:** the two ciphertexts differ. Transit uses a fresh nonce each time, so equal plaintexts cannot be spotted in the database. (If you need equal ciphertexts — for example to look rows up — the study guide covers `convergent_encryption`.)

**Step 3 — Rotate, then rewrap.**

```
$ vault write -f transit/keys/orders/rotate
$ vault write -field=ciphertext transit/encrypt/orders plaintext=$(echo -n "5500 0000 0000
0004" | base64)
$ vault write -field=plaintext transit/decrypt/orders ciphertext=$CT1 | base64 -d; echo
$ vault write -field=ciphertext transit/rewrap/orders ciphertext=$CT1
```

Lab 3.2 · Step 3 · Rotate and rewrap — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault write -f transit/keys/orders/rotate
Key                     Value
---                     -----
latest_version          2
min_decryption_version  1
name                    orders
trainee@vault-lab:~$ vault write -field=ciphertext transit/encrypt/orders plaintext=$(echo -n
                       "5500 0000 0000 0004" | base64)
vault:v2:3K1wqk2v8pH0rS2mQ0d5yN1bWc8xA7fU9eLzT4iJ6gR2oV0hY5sP1A==
trainee@vault-lab:~$ vault write -field=plaintext transit/decrypt/orders ciphertext=$CT1 |
                       base64 -d; echo
4111 1111 1111 1111
trainee@vault-lab:~$ vault write -field=ciphertext transit/rewrap/orders ciphertext=$CT1
vault:v2:Lr7mP0qV2s1Xc9dA4fK6hT3yB8nW5eJ0uZ2iQ7oG1bS4vN9xR3cE6g==
```

*Expected output — key at version 2; new data is v2; old v1 data still decrypts; rewrap upgrades it to v2*

Rewrap never shows the plaintext to the caller, so a batch job can upgrade a whole table with a policy that has no `decrypt` right.

**Step 4 — Retire old versions.** After all data is rewrapped, raise the floor so a leaked old key version is useless.

```
$ vault write transit/keys/orders/config min_decryption_version=2
$ vault write transit/decrypt/orders ciphertext=$CT1
```

**Step 5 — Generate a data key for large files (envelope encryption).**

```
$ vault write -f transit/datakey/plaintext/orders
```

Lab 3.2 · Steps 4-5 · Decryption floor and data keys — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault write transit/keys/orders/config min_decryption_version=2
trainee@vault-lab:~$ vault write transit/decrypt/orders ciphertext=$CT1
Error writing data to transit/decrypt/orders: Error making API request.

URL: PUT http://127.0.0.1:8200/v1/transit/decrypt/orders
Code: 400. Errors:

* ciphertext or signature version is disallowed by policy (too old)
trainee@vault-lab:~$ vault write -f transit/datakey/plaintext/orders
Key            Value
---            -----
ciphertext     vault:v2:lZ6g1nZr0F2pQ8bT4xW9mE3kJ7sY1cV5hA0uD2oR6iN8...
key_version    2
plaintext      m3JkV1o2QWg3cXhTRzlqTnBkUmJ3YzRvRWx5VUt0aE0=
```

*Expected output — v1 ciphertext refused as too old; a data key returned in plaintext and wrapped form*

The app encrypts the file locally with `plaintext` (a 256-bit key), stores only `ciphertext` next to the file, and discards the plaintext key. To read the file later it asks Transit to decrypt the data key. Use `transit/datakey/wrapped/orders` when the caller should never see the plaintext key at creation time.

**Step 6 — Sign and verify (optional).**

```
$ vault write transit/keys/release-signing type=ed25519
$ SIG=$(vault write -field=signature transit/sign/release-signing input=$(echo -n "build-2026.10.16" | base64))
$ vault write transit/verify/release-signing input=$(echo -n "build-2026.10.16" | base64) signature=$SIG
```

✓ **Checkpoint:** `valid true`. Change one character of the input and verify again: `valid false`.

**Step 7 — Least-privilege policy for the app.**

```
# orders-app.hcl
path "transit/encrypt/orders" {
  capabilities = ["update"]
}
path "transit/decrypt/orders" {
  capabilities = ["update"]
}
```

```
$ vault policy write orders-app orders-app.hcl
$ APP=$(vault token create -policy=orders-app -ttl=1h -field=token)
$ VAULT_TOKEN=$APP vault write transit/encrypt/orders plaintext=$(echo -n test | base64)
$ VAULT_TOKEN=$APP vault read transit/keys/orders
$ VAULT_TOKEN=$APP vault write -f transit/keys/orders/rotate
```

✓ **Checkpoint:** the encrypt call works; reading the key and rotating it both return `403 permission denied`. The app can use the key but never manage it.

## Clean up

Keep the dev server running for Lab 3.4. Lab 3.10 creates its own `orders` key on a fresh server.

# Lab 3.3 — Transit auto-unseal and recovery keys

**Objective:** protect a second Vault server with the Transit seal of the dev server, initialise it with recovery keys, prove it unseals itself after a restart, and see what a broken unsealer looks like. **Time:** 25 min. **Study guide:** Day 3, Module 3. **Environment:** the dev server (acts as the *unsealer*, port 8200) plus a new *seal node* on port 8210.

Cloud KMS auto-unseal (AWS KMS, Azure Key Vault, GCP Cloud KMS) works exactly the same way; only the `seal` stanza changes (study guide, Module 3). Transit is used here because it needs no cloud account.

## Steps

**Step 1 — Create the unseal key on the dev server.** In terminal 2 ( `VAULT_ADDR=http://127.0.0.1:8200` , `VAULT_TOKEN=root` ). If you skipped Lab 3.2, run `vault secrets enable transit` first.

```
$ mkdir -p ~/day3/seal/data && cd ~/day3/seal
$ vault write -f transit/keys/autounseal
```

**Step 2 — Give the seal node a narrow, long-lived token.** It may only encrypt and decrypt with this one key. The token is orphan (it survives its parent's revocation) and periodic (it renews forever while in use).

```
# autounseal.hcl
path "transit/encrypt/autounseal" {
  capabilities = ["update"]
}
path "transit/decrypt/autounseal" {
  capabilities = ["update"]
}
```

```
$ vault policy write autounseal autounseal.hcl
$ vault token create -orphan -policy=autounseal -period=24h -field=token >
~/day3/seal/unseal-token
$ vault token lookup $(cat ~/day3/seal/unseal-token) | grep -E "orphan|period|policies"
```

Lab 3.3 · Steps 1-2 · Prepare the unsealer — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault write -f transit/keys/autounseal
Success! Data written to: transit/keys/autounseal
trainee@vault-lab:~$ vault policy write autounseal autounseal.hcl
Success! Uploaded policy: autounseal
trainee@vault-lab:~$ vault token create -orphan -policy=autounseal -period=24h -field=token >
                     ~/day3/seal/unseal-token
trainee@vault-lab:~$ vault token lookup $(cat ~/day3/seal/unseal-token) | grep -E
                     "orphan|period|policies"
orphan              true
period              24h
policies            [autounseal default]
```

*Expected output — key autounseal created; orphan periodic token with only the autounseal policy*

**Step 3 — Write the seal node's configuration.** The unquoted `EOF` lets the shell fill in `$HOME` .

```
$ cat > ~/day3/seal/seal.hcl <<EOF
ui            = true
disable_mlock = true
```

```
api_addr      = "http://127.0.0.1:8210"
cluster_addr  = "https://127.0.0.1:8211"

storage "raft" {
  path    = "$HOME/day3/seal/data"
  node_id = "seal1"
}

listener "tcp" {
  address         = "127.0.0.1:8210"
  cluster_address = "127.0.0.1:8211"
  tls_disable     = true
}

seal "transit" {
  address    = "http://127.0.0.1:8200"
  key_name   = "autounseal"
  mount_path = "transit/"
  # the token comes from VAULT_TOKEN when the server starts
}
EOF
```

**Step 4 — Start and initialise the seal node.** Use terminal 3 so terminal 2 keeps talking to the dev server.

```
$ cd ~/day3/seal && VAULT_TOKEN=$(cat unseal-token) vault server -config=seal.hcl > seal.log
2>&1 &
$ export VAULT_ADDR=http://127.0.0.1:8210
$ vault operator init -recovery-shares=3 -recovery-threshold=2
```

Lab 3.3 · Step 4 · Start and initialise the seal node (terminal 3) — EXPECTED OUTPUT

```
trainee@vault-lab:~$ cd ~/day3/seal && VAULT_TOKEN=$(cat unseal-token) vault server
                       -config=seal.hcl > seal.log 2>&1 &
[1] 48213
trainee@vault-lab:~$ export VAULT_ADDR=http://127.0.0.1:8210
trainee@vault-lab:~$ vault operator init -recovery-shares=3 -recovery-threshold=2
Recovery Key 1: 5qkB6wO2cN0yP1+eR4hJx3tL9mV7aF2sD8uK0iG5zQ1T
Recovery Key 2: Hn3rT8yW1bV6cM0xZ4pL7kJ2gF9dS5aQ1eU0iO8wR3Y6
Recovery Key 3: p0Lk9jH8gF7dS6aQ5wE4rT3yU2iO1pZ0xC9vB8nM7qW6
Initial Root Token: hvs.Q5d9WlqA2p0o7bVh3FzXr1Kc
Success! Vault is initialized
Recovery key initialized with 3 key shares and a key threshold of 2. Please
securely distribute the key shares printed above.
```

*Expected output — initialised with three recovery keys (threshold 2) and a root token*

There are no unseal keys: the root key is encrypted by the Transit key on the dev server. The recovery keys cannot unseal anything; they authorise sensitive operations such as `vault operator generate-root` and `vault operator rekey`. Save them to `~/day3/seal/recovery.txt` (lab only).

**Step 5 — Restart without unsealing.**

```
$ kill %1; sleep 2
$ VAULT_TOKEN=$(cat unseal-token) vault server -config=seal.hcl > seal.log 2>&1 &
$ sleep 5; vault status
$ grep -E "unseal" seal.log | tail -2
```

Lab 3.3 · Step 5 · Restart: no unseal needed — EXPECTED OUTPUT

```
trainee@vault-lab:~$ kill %1; sleep 2
trainee@vault-lab:~$ VAULT_TOKEN=$(cat unseal-token) vault server -config=seal.hcl > seal.log
                     2>&1 &
trainee@vault-lab:~$ sleep 5; vault status
Key                      Value
---                      -----
Seal Type                transit
Recovery Seal Type       shamir
Initialized              true
Sealed                   false
Total Recovery Shares    3
Threshold                2
Version                  2.1.2
Storage Type             raft
HA Enabled               true
trainee@vault-lab:~$ grep -E "unseal" seal.log | tail -2
2026-10-16T10:58:31.402+0530 [INFO]  core: stored unseal keys supported, attempting fetch
2026-10-16T10:58:31.418+0530 [INFO]  core: vault is unsealed
```

*Expected output — Seal Type transit, Recovery Seal Type shamir, Sealed false straight after start*

✓ **Checkpoint:** `Sealed false` with no `vault operator unseal` typed, and the log says `vault is unsealed`.

**Step 6 — Break the unsealer.** Take away the decrypt right and restart the seal node.

```
$ VAULT_ADDR=http://127.0.0.1:8200 vault policy write autounseal - <<< 'path
"transit/encrypt/autounseal" { capabilities = ["update"] }'
$ kill %1; sleep 2; VAULT_TOKEN=$(cat unseal-token) vault server -config=seal.hcl > seal.log
2>&1 &
$ sleep 5; vault status | grep -E "^Sealed"
$ grep -i "error" seal.log | tail -1 | cut -c1-140
```

Lab 3.3 · Step 6 · A broken unsealer — EXPECTED OUTPUT

```
trainee@vault-lab:~$ VAULT_ADDR=http://127.0.0.1:8200 vault policy write autounseal - <<< 'path
                     "transit/encrypt/autounseal" { capabilities = ["update"] }'
trainee@vault-lab:~$ kill %1; sleep 2; VAULT_TOKEN=$(cat unseal-token) vault server
                     -config=seal.hcl > seal.log 2>&1 &
trainee@vault-lab:~$ sleep 5; vault status | grep -E "^Sealed"
Sealed                   true
trainee@vault-lab:~$ grep -i "error" seal.log | tail -1 | cut -c1-140
2026-10-16T11:01:12.774+0530 [ERROR] core: failed to unseal: error="... Code: 403. Errors: *
    permission denied"
Restore autounseal.hcl (Step 2); the server retries and unseals on its own.
```

*Expected output — the node stays sealed; the log shows the 403 from the unsealer*

This is the first thing to check in production when a node will not unseal: can it reach the KMS, and does its identity still have encrypt/decrypt on the key? (For AWS: `kms:Encrypt`, `kms:Decrypt`, `kms:DescribeKey`.)

**Step 7 — Repair.** Restore the policy; the seal node retries on its own.

```
$ VAULT_ADDR=http://127.0.0.1:8200 vault policy write autounseal autounseal.hcl
$ sleep 10; vault status | grep -E "^Sealed"
```

✓ **Checkpoint:** `Sealed false`. If it is still sealed after 30 seconds, restart the seal node as in Step 5.

**Step 8 — Discuss: seal migration and seal wrap.**

- To move an existing Shamir cluster (such as the Lab 1.3 server) to auto-unseal: add the `seal` stanza, restart, then run `vault operator unseal -migrate` with the old unseal keys until the threshold is met. The old unseal keys become the new recovery keys.
- The dev server holding `autounseal` is now critical: if it loses the key, the seal node can never unseal again. In production the unsealing cluster is itself HA, backed up and monitored.
- **Seal wrap (Enterprise)** adds a second layer: selected values (for example PKI CA keys, or a mount enabled with `-seal-wrap`) are encrypted again with the seal's KMS/HSM key before they reach the barrier, which supports FIPS 140-2/3 key-storage requirements.

## Clean up

Ports 8210/8211 are needed in Lab 3.5.

```
$ kill %1; unset VAULT_ADDR; rm -rf ~/day3/seal
$ export VAULT_ADDR=http://127.0.0.1:8200 VAULT_TOKEN=root
```

# Lab 3.4 — Vault Agent: templates, pkiCert, exec and API proxy

**Objective:** run Vault Agent with AppRole auto-auth; render a config file from KV and a TLS certificate from PKI; re-render and run a command when the secret changes; run an app as a supervised child process with secrets in its environment; and call Vault through the Agent's API proxy without a token. **Time:** 35 min. **Study guide:** Day 3, Module 4 (Day 2 Lab 2.4 introduced Agent with a single template). **Environment:** dev server with the `pki_int` mount and `web-server` role from Lab 3.1.

## Steps

**Step 1 — A secret, a policy and an AppRole.** In terminal 2:

```
$ mkdir -p ~/day3/agent && cd ~/day3/agent
$ vault kv put secret/webapp db_user=webapp db_password=S3cure-v1
```

```
# agent-demo.hcl
path "secret/data/webapp" {
  capabilities = ["read"]
}
path "pki_int/issue/web-server" {
  capabilities = ["update"]
}
```

```
$ vault policy write agent-demo agent-demo.hcl
$ vault auth enable approle
$ vault write auth/approle/role/agent-demo token_policies=agent-demo token_ttl=1h
token_max_ttl=4h
$ vault read -field=role_id auth/approle/role/agent-demo/role-id > role_id
$ vault write -f -field=secret_id auth/approle/role/agent-demo/secret-id > secret_id
$ chmod 600 role_id secret_id
```

**Step 2 — Two templates.** The first renders a properties file from KV; the second issues a certificate with `pkiCert` and also writes the key and the certificate chain to their own files.

```
$ cat > app.properties.ctmpl <<'EOF'
# rendered by Vault Agent - do not edit
{{ with secret "secret/data/webapp" -}}
db.user={{ .Data.data.db_user }}
db.password={{ .Data.data.db_password }}
{{- end }}
EOF

$ cat > web.pem.ctmpl <<EOF
{{- with pkiCert "pki_int/issue/web-server" "common_name=app.stackmaster.lab" "ttl=1h" -}}
{{ .Cert }}{{ .CA }}{{ .Key }}
{{ .Key | writeToFile "$HOME/day3/agent/web.key" "" "" "0600" }}
{{ print .Cert "\n" .CA | writeToFile "$HOME/day3/agent/web.crt" "" "" "0644" }}
{{- end -}}
EOF
```

`pkiCert` uses the destination file (`web.pem`) as its cache: on restart, Agent reads the certificate in it and only issues a new one when it is close to expiry. That is why the destination must contain `.Cert`.

**Step 3 — The Agent configuration.**

```
$ cat > agent.hcl <<EOF
pid_file = "$HOME/day3/agent/agent.pid"

vault {
  address = "http://127.0.0.1:8200"
}

auto_auth {
  method "approle" {
    config = {
      role_id_file_path                   = "$HOME/day3/agent/role_id"
      secret_id_file_path                 = "$HOME/day3/agent/secret_id"
      remove_secret_id_file_after_reading = false
    }
  }
  sink "file" {
    config = {
      path = "$HOME/day3/agent/agent-token"
      mode = 0600
    }
  }
}

template_config {
  static_secret_render_interval = "15s"   # how often KV is re-read (default 5m)
  exit_on_retry_failure         = true
}

template {
  source      = "$HOME/day3/agent/app.properties.ctmpl"
  destination = "$HOME/day3/agent/app.properties"
  perms       = "0600"
  exec {
    command = ["sh", "-c", "echo reloaded >> reload.log"]
  }
}

template {
  source      = "$HOME/day3/agent/web.pem.ctmpl"
  destination = "$HOME/day3/agent/web.pem"
  perms       = "0600"
}

cache {}
api_proxy {
  use_auto_auth_token = true
}
listener "tcp" {
  address     = "127.0.0.1:8100"
  tls_disable = true
}
EOF
```

`remove_secret_id_file_after_reading = false` keeps the lab repeatable; in production leave it at the default (`true`) and deliver the SecretID with response wrapping (Day 2 Lab 2.4).

**Step 4 — Run Agent.** In terminal 3:

```
$ cd ~/day3/agent && vault agent -config=agent.hcl
```

Lab 3.4 · Step 4 · vault agent (terminal 3) EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault agent -config=agent.hcl
==> Vault Agent started! Log data will stream in below:

==> Vault Agent configuration:

           Api Address 1: http://127.0.0.1:8100
                    Cgo: disabled
                Log Level: info
                  Version: Vault v2.1.2, built 2026-09-24T10:11:07Z

[INFO]  agent.sink.file: creating file sink
[INFO]  agent.auth.handler: starting auth handler
[INFO]  agent.auth.handler: authenticating
[INFO]  agent.auth.handler: authentication successful, sending token to sinks
[INFO]  agent.sink.file: token written: path=/home/trainee/day3/agent/agent-token
[INFO]  agent.template.server: starting template server
[INFO] (runner) creating new runner (dry: false, once: false)
[INFO] (runner) rendered "(dynamic)" => "/home/trainee/day3/agent/app.properties"
[INFO] (runner) rendered "(dynamic)" => "/home/trainee/day3/agent/web.pem"
```

*Expected output — AppRole login succeeds, token written to the sink, both templates rendered*

**Step 5 — Check the rendered files.** In terminal 2:

```
$ cat ~/day3/agent/app.properties
$ ls -l ~/day3/agent/
$ openssl x509 -in ~/day3/agent/web.crt -noout -subject -enddate
```

Lab 3.4 · Step 5 · Rendered files EXPECTED OUTPUT

```
trainee@vault-lab:~$ cat ~/day3/agent/app.properties
# rendered by Vault Agent - do not edit
db.user=webapp
db.password=S3cure-v1
trainee@vault-lab:~$ ls -l ~/day3/agent/
-rw------- 1 trainee trainee   29 Oct 16 11:12 agent-token
-rw------- 1 trainee trainee   88 Oct 16 11:12 app.properties
-rw-r--r-- 1 trainee trainee 2581 Oct 16 11:12 web.crt
-rw------- 1 trainee trainee 1675 Oct 16 11:12 web.key
-rw------- 1 trainee trainee 5937 Oct 16 11:12 web.pem
trainee@vault-lab:~$ openssl x509 -in ~/day3/agent/web.crt -noout -subject -enddate
subject=CN = app.stackmaster.lab
notAfter=Oct 16 12:12:31 2026 GMT
```

*Expected output — properties file holds the password; key files are 0600; certificate valid for 1 hour*

✓ **Checkpoint:** `db.password=S3cure-v1`, `web.key` is mode 0600, and the certificate's `notAfter` is one hour away.

**Step 6 — Change the secret and watch Agent react.**

```
$ vault kv put secret/webapp db_user=webapp db_password=S3cure-v2
$ sleep 20; cat ~/day3/agent/app.properties
$ cat ~/day3/agent/reload.log
```

Lab 3.4 · Step 6 · Change the secret, watch it re-render — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault kv put secret/webapp db_user=webapp db_password=S3cure-v2
==== Secret Path ====
secret/data/webapp
...
version            2
trainee@vault-lab:~$ sleep 20; cat ~/day3/agent/app.properties
# rendered by Vault Agent - do not edit
db.user=webapp
db.password=S3cure-v2
terminal 3 (Agent log):
[INFO] (runner) rendered "(dynamic)" => "/home/trainee/day3/agent/app.properties"
[INFO] (runner) executing command "[\"sh\", \"-c\", \"echo reloaded >> reload.log\"]" from
    "(dynamic)" => "/home/trainee/day3/agent/app.properties"
```

*Expected output — the file re-renders with S3cure-v2 and Agent runs the exec command*

In real use the `exec` command reloads the service (`systemctl reload nginx`, `kill -HUP <pid>`). Lab 3.10 uses it to reload nginx when the certificate is renewed.

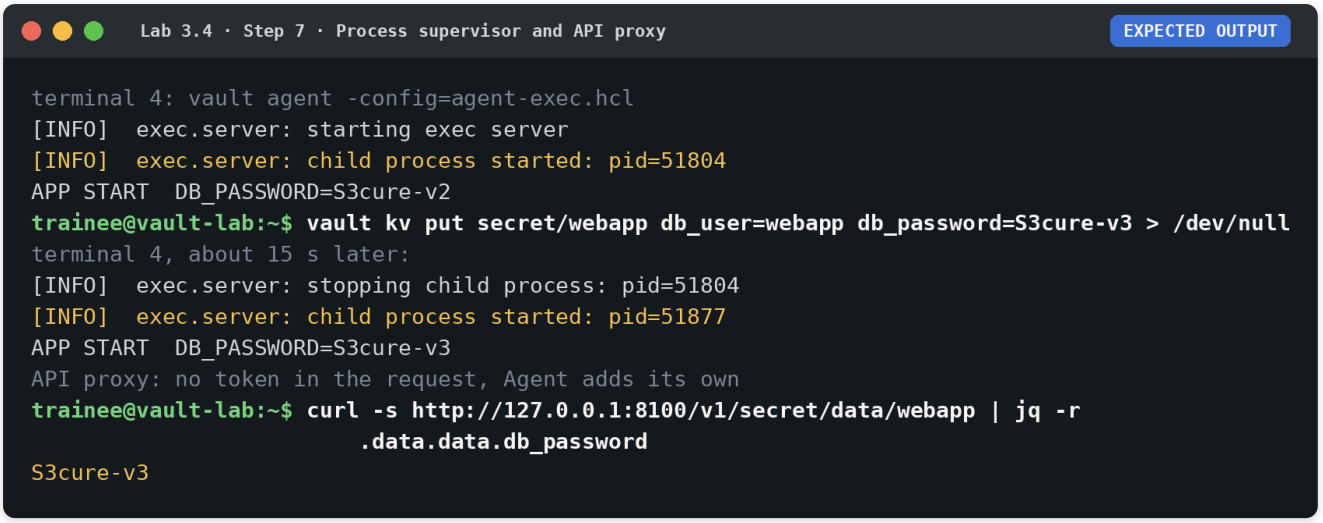
**Step 7 — Process supervisor mode.** Instead of files, Agent starts the app itself with secrets as environment variables, and restarts it when they change. In terminal 4:

```
$ cd ~/day3/agent && cat > agent-exec.hcl <<EOF
vault {
  address = "http://127.0.0.1:8200"
}
auto_auth {
  method "approle" {
    config = {
      role_id_file_path                   = "$HOME/day3/agent/role_id"
      secret_id_file_path                 = "$HOME/day3/agent/secret_id"
      remove_secret_id_file_after_reading = false
    }
  }
}
template_config {
  static_secret_render_interval = "10s"
}
env_template "DB_PASSWORD" {
  contents             = "{{ with secret \"secret/data/webapp\" }}{{ .Data.data.db_password
}}{{ end }}"
  error_on_missing_key = true
}
exec {
  command                   = ["sh", "-c", "echo \"APP START  DB_PASSWORD=\$DB_PASSWORD\";
sleep 3600"]
  restart_on_secret_changes = "always"
  restart_stop_signal       = "SIGTERM"
}
EOF
$ vault agent -config=agent-exec.hcl
```

Then in terminal 2 change the secret again and call Vault through the first Agent's API proxy — note there is no `X-Vault-Token` header:

```
$ vault kv put secret/webapp db_user=webapp db_password=S3cure-v3 > /dev/null
```

```
$ curl -s http://127.0.0.1:8100/v1/secret/data/webapp | jq -r .data.data.db_password
```




*Expected output — the child process restarts with S3cure-v3; the proxy answers without a client token*

✓ **Checkpoint:** terminal 4 shows a new child PID with `S3cure-v3`, and the curl prints `S3cure-v3`. The app is printing its secret only to make the lab visible — never log secrets in real code.

**Step 8 — Think about it.** Which of the three patterns would you use for: nginx TLS certificates; a Java app that reads `application.properties` at start-up; a legacy binary that only reads environment variables; a script that needs Transit encryption? (Answers: template + exec reload; template; env_template + exec; API proxy.)

## Clean up

Press Ctrl+C in terminals 3 and 4. Keep `~/day3/agent` for reference. The dev server can keep running; Lab 3.6 restarts it with a telemetry configuration.

# Lab 3.5 — Three-node Raft: failover and snapshots

**Objective:** run a three-node Integrated Storage (Raft) cluster on one machine, watch a follower take over when the leader fails, check health with autopilot, and recover deleted data from a snapshot. **Time:** 35 min. **Study guide:** Day 3, Module 5. **Environment:** Raft cluster (three local processes on ports 8210, 8220, 8230; cluster ports +1). The dev server on 8200 may keep running. Lab 3.3 must be cleaned up first (it used 8210).

## Steps

**Step 1 — The cluster script.** It writes one config per node (each with `retry_join` blocks pointing at all three) and starts or stops nodes.

```
#!/usr/bin/env bash
# ~/day3/raft-cluster.sh — start/stop a 3-node Vault Raft cluster on one machine (lab only)
set -euo pipefail
BASE=$HOME/day3/raft
NODES=(node1:8210 node2:8220 node3:8230)

conf() {   # $1 = node name, $2 = API port
  local n=$1 p=$2 c=$(( $2 + 1 ))
  mkdir -p "$BASE/$n/data"
  cat > "$BASE/$n/config.hcl" <<EOF
ui            = true
disable_mlock = true
api_addr      = "http://127.0.0.1:$p"
cluster_addr  = "https://127.0.0.1:$c"
listener "tcp" {
  address         = "127.0.0.1:$p"
  cluster_address = "127.0.0.1:$c"
  tls_disable     = true
}
storage "raft" {
  path    = "$BASE/$n/data"
  node_id = "$n"
  retry_join { leader_api_addr = "http://127.0.0.1:8210" }
  retry_join { leader_api_addr = "http://127.0.0.1:8220" }
  retry_join { leader_api_addr = "http://127.0.0.1:8230" }
}
EOF
}

start() {
  for e in "${NODES[@]}"; do
    n=${e%%:*}; p=${e##*:}
    [[ -n "${1:-}" && "$1" != "$n" ]] && continue
    [[ -f "$BASE/$n/config.hcl" ]] || conf "$n" "$p"
    vault server -config="$BASE/$n/config.hcl" > "$BASE/$n/vault.log" 2>&1 &
    echo $! > "$BASE/$n/vault.pid"
    echo "started $n on 127.0.0.1:$p (pid $!)"
  done
}

stop() {
  for e in "${NODES[@]}"; do
    n=${e%%:*}
    [[ -n "${1:-}" && "$1" != "$n" ]] && continue
    if [[ -f "$BASE/$n/vault.pid" ]]; then
      kill "$(cat "$BASE/$n/vault.pid")" 2>/dev/null && echo "stopped $n (pid $(cat
"$BASE/$n/vault.pid"))"
```

```
      rm -f "$BASE/$n/vault.pid"
    fi
  done
}

case "${1:-}" in
  start)   start "${2:-}" ;;
  stop)    stop "${2:-}" ;;
  destroy) stop; rm -rf "$BASE" ;;
  *)       echo "usage: $0 start|stop [node1|node2|node3] | destroy"; exit 1 ;;
esac
```

```
$ cd ~/day3 && chmod +x raft-cluster.sh
```

**Step 2 — Start, initialise once, unseal all three.** Only node1 is initialised; node2 and node3 join it through `retry_join` when they are unsealed. One key share is a lab shortcut — production uses 5/3 or auto-unseal (Lab 3.3).

```
$ ./raft-cluster.sh start
$ export VAULT_ADDR=http://127.0.0.1:8210
$ vault operator init -key-shares=1 -key-threshold=1 -format=json > ~/day3/raft/init.json
$ KEY=$(jq -r '.unseal_keys_b64[0]' ~/day3/raft/init.json)
$ for p in 8210 8220 8230; do VAULT_ADDR=http://127.0.0.1:$p vault operator unseal $KEY >
/dev/null; sleep 3; done
$ export VAULT_TOKEN=$(jq -r .root_token ~/day3/raft/init.json)
$ vault operator raft list-peers
```

Lab 3.5 · Step 2 · Three-node Raft cluster — EXPECTED OUTPUT

```
trainee@vault-lab:~$ ./raft-cluster.sh start
started node1 on 127.0.0.1:8210 (pid 52110)
started node2 on 127.0.0.1:8220 (pid 52111)
started node3 on 127.0.0.1:8230 (pid 52112)
trainee@vault-lab:~$ export VAULT_ADDR=http://127.0.0.1:8210
trainee@vault-lab:~$ vault operator init -key-shares=1 -key-threshold=1 -format=json >
                     ~/day3/raft/init.json
trainee@vault-lab:~$ KEY=$(jq -r '.unseal_keys_b64[0]' ~/day3/raft/init.json)
trainee@vault-lab:~$ for p in 8210 8220 8230; do VAULT_ADDR=http://127.0.0.1:$p vault operator
                     unseal $KEY > /dev/null; done
trainee@vault-lab:~$ export VAULT_TOKEN=$(jq -r .root_token ~/day3/raft/init.json)
trainee@vault-lab:~$ vault operator raft list-peers
Node     Address           State       Voter
----     -------           -----       -----
node1    127.0.0.1:8211    leader      true
node2    127.0.0.1:8221    follower    true
node3    127.0.0.1:8231    follower    true
```

*Expected output — three voters: node1 leader, node2 and node3 followers*

✓ **Checkpoint:** three rows, all `Voter true`, exactly one `leader`. If a node is missing, read its log (`~/day3/raft/node2/vault.log`) and unseal it again.

**Step 3 — Health, and some data to protect.**

```
$ vault operator raft autopilot state
$ vault secrets enable -path=secret kv-v2
```

```
$ vault kv put secret/cluster-test owner=vivek
$ VAULT_ADDR=http://127.0.0.1:8230 vault kv get -field=owner secret/cluster-test
```

Lab 3.5 · Step 3 · Autopilot health — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault operator raft autopilot state
Healthy:                        true
Failure Tolerance:              1
Leader:                         node1
Voters:
   node1
   node2
   node3
Servers:
   node1
      Name:             node1
      Address:          127.0.0.1:8211
      Status:           leader
      Node Status:      alive
      Healthy:          true
      Last Contact:     0s
      Last Term:        3
      Last Index:       61
   node2
      Name:             node2
      Status:           voter
      Healthy:          true
      Last Contact:     2.31ms
   ...
```

*Expected output — autopilot healthy with failure tolerance 1*

A non-dev server has no `secret/` mount, so you enable it. The last command reads through a standby: standbys forward requests to the leader, so clients can use any node.

**Step 4 — Fail the leader.**

```
$ ./raft-cluster.sh stop node1
$ export VAULT_ADDR=http://127.0.0.1:8220
$ sleep 10; vault status | grep -E "HA Mode|Active Node"
$ vault kv get -field=owner secret/cluster-test
$ vault operator raft list-peers
$ vault operator raft autopilot state | head -2
```

Lab 3.5 · Step 4 · Fail the leader — EXPECTED OUTPUT

```
trainee@vault-lab:~$ ./raft-cluster.sh stop node1
stopped node1 (pid 52110)
trainee@vault-lab:~$ export VAULT_ADDR=http://127.0.0.1:8220
trainee@vault-lab:~$ sleep 10; vault status | grep -E "HA Mode|Active Node"
HA Mode                standby
Active Node Address    http://127.0.0.1:8230
trainee@vault-lab:~$ vault kv get -field=owner secret/cluster-test
vivek
trainee@vault-lab:~$ vault operator raft list-peers
Node     Address           State       Voter
----     -------           -----       -----
node1    127.0.0.1:8211    follower    true
node2    127.0.0.1:8221    follower    true
node3    127.0.0.1:8231    leader      true
trainee@vault-lab:~$ vault operator raft autopilot state | head -2
Healthy:                       false
Failure Tolerance:             0
```

*Expected output — node3 (or node2) is the new leader; data still readable; failure tolerance 0*

✓ **Checkpoint:** a new leader was elected within seconds and the secret is still readable. Autopilot reports `Healthy false` and `Failure Tolerance 0`: one more failure would stop writes.

**Step 5 — Bring node1 back.**

```
$ ./raft-cluster.sh start node1
$ VAULT_ADDR=http://127.0.0.1:8210 vault operator unseal $KEY > /dev/null
$ sleep 5; vault operator raft list-peers
$ vault operator raft autopilot state | head -2
```

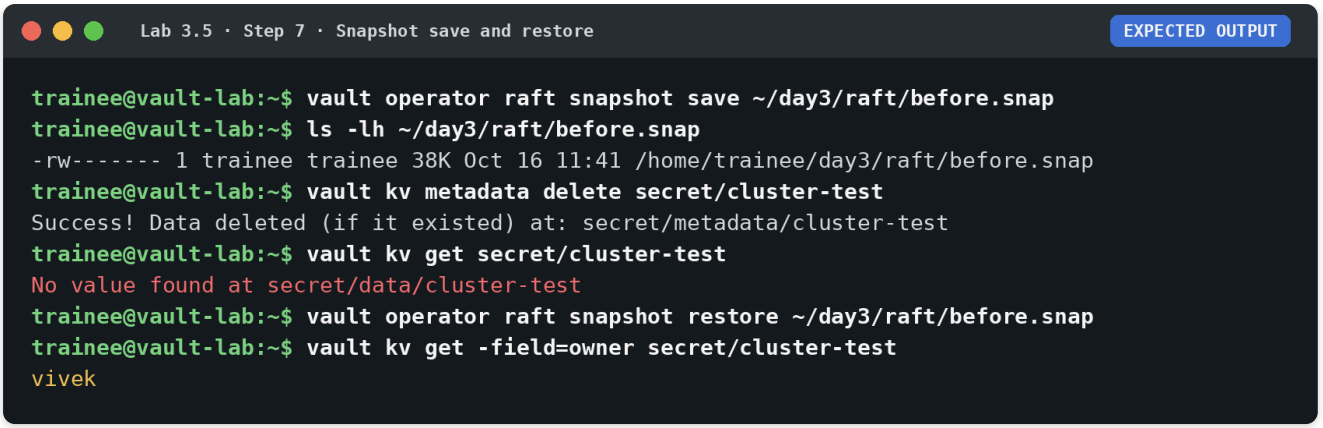
node1 rejoins as a follower and catches up from the leader's log; autopilot returns to `Healthy true`, `Failure Tolerance 1`.

**Step 6 — Planned leadership change.** Before maintenance on the leader, hand leadership over instead of killing it:

```
$ LEADER=$(vault operator raft list-peers -format=json | jq -r '.data.config.servers[] |
select(.leader) | .node_id')
$ echo "leader is $LEADER"
$ vault operator step-down          # sent to the active node through any node
$ sleep 3; vault operator raft list-peers
```

**Step 7 — Snapshot, delete, restore.**

```
$ vault operator raft snapshot save ~/day3/raft/before.snap
$ ls -lh ~/day3/raft/before.snap
$ vault kv metadata delete secret/cluster-test
$ vault kv get secret/cluster-test
$ vault operator raft snapshot restore ~/day3/raft/before.snap
$ vault kv get -field=owner secret/cluster-test
```




*Expected output — snapshot saved; secret deleted; restored from the snapshot*

A restore rolls the **whole cluster** back to the moment of the snapshot — every secret, policy, token and lease. Use it for disaster recovery, not to undo one change (KV v2 versioning from Day 1 is the tool for that). `-force` is only needed to restore a snapshot taken from a different cluster (its keys must match).

**Step 8 — Discuss: losing quorum.** With 3 nodes you can lose 1; with 5 you can lose 2. If you stopped node2 *and* node3 now, node1 alone could not elect itself. The study guide (Module 7.4) describes the two recoveries: a `peers.json` file on the survivor, or a new cluster plus the latest snapshot. Automate snapshots (Lab 3.8 does it with Ansible) and copy them off the cluster.

## Clean up

Keep the cluster for **Lab 3.8**. To pause it: `./raft-cluster.sh stop`. To resume: `./raft-cluster.sh start`, then unseal each node with `$KEY` (from `init.json`). To delete it after Lab 3.8: `./raft-cluster.sh destroy`. When you return to the dev server, run `export VAULT_ADDR=http://127.0.0.1:8200 VAULT_TOKEN=root`.

# Lab 3.6 — Telemetry, Prometheus and audit devices

**Objective:** expose Vault metrics in Prometheus format, scrape them with a metrics-only token, enable two audit devices, trace who read a secret, and match a known value against its HMAC. **Time:** 25 min. **Study guide:** Day 3, Module 6. **Environment:** dev server restarted with a telemetry file, plus Docker for Prometheus.

## Steps

**Step 1 — Restart the dev server with telemetry.** Press Ctrl+C in terminal 1 (Labs 3.1–3.4 are finished; their data goes with it). A dev server merges any `-config` file into its built-in settings.

```
$ mkdir -p ~/day3/mon && cd ~/day3/mon
$ cat > telemetry.hcl <<'EOF'
telemetry {
  prometheus_retention_time = "24h"
  disable_hostname          = true
}
EOF
$ vault server -dev -dev-root-token-id=root -config=$HOME/day3/mon/telemetry.hcl
```

**Step 2 — A metrics-only token, and a first look.** In terminal 2 ( `VAULT_ADDR=http://127.0.0.1:8200` , `VAULT_TOKEN=root` ):

```
$ cd ~/day3/mon
$ vault policy write prometheus-metrics - <<< 'path "sys/metrics" { capabilities = ["read"]
}'
$ vault token create -policy=prometheus-metrics -period=24h -field=token > ~/day3/mon/prom-
token
$ chmod 644 prom-token      # lab only: the Prometheus container runs as another user
$ curl -s -H "X-Vault-Token: $(cat ~/day3/mon/prom-token)" \
    "$VAULT_ADDR/v1/sys/metrics?format=prometheus" | grep -E
"^vault_(core_unsealed|core_active|token_count|expire_num_leases) "
$ curl -s -o /dev/null -w "%{http_code}\n" $VAULT_ADDR/v1/sys/health
```

Lab 3.6 · Step 2 · Prometheus metrics from Vault — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault policy write prometheus-metrics - <<< 'path "sys/metrics" {
                       capabilities = ["read"] }'
Success! Uploaded policy: prometheus-metrics
trainee@vault-lab:~$ vault token create -policy=prometheus-metrics -period=24h -field=token >
                       ~/day3/mon/prom-token
trainee@vault-lab:~$ curl -s -H "X-Vault-Token: $(cat ~/day3/mon/prom-token)" \
    "$VAULT_ADDR/v1/sys/metrics?format=prometheus" | grep -E
    "^vault_(core_unsealed|core_active|token_count|expire_num_leases) "
vault_core_active 1
vault_expire_num_leases 3
vault_core_unsealed 1
vault_token_count 6
trainee@vault-lab:~$ curl -s -o /dev/null -w "%{http_code}\n" $VAULT_ADDR/v1/sys/health
200
```

*Expected output — core_unsealed 1, core_active 1, token and lease counts; health returns 200*

The counts vary. Run `vault read sys/metrics` to see everything in JSON, and compare the names with the alert table in the study guide (Module 6.2).

**Step 3 — Scrape with Prometheus.**

```
# ~/day3/mon/prometheus.yml
global:
  scrape_interval: 10s
scrape_configs:
  - job_name: vault
    metrics_path: /v1/sys/metrics
    params:
      format: [prometheus]
    authorization:
      credentials_file: /etc/prometheus/prom-token
    static_configs:
      - targets: ["127.0.0.1:8200"]
```

```
$ docker run -d --name prom --network host -v ~/day3/mon:/etc/prometheus prom/prometheus
$ sleep 15; curl -s localhost:9090/api/v1/targets | jq -r '.data.activeTargets[] | "\
(.labels.job) \(.health)"'
$ curl -s 'localhost:9090/api/v1/query?query=vault_core_unsealed' | jq -r '.data.result[] |
"\(.metric.instance) \(.value[1])"'
```

Lab 3.6 · Step 3 · Prometheus scrapes Vault — EXPECTED OUTPUT

```
trainee@vault-lab:~$ docker run -d --name prom --network host -v ~/day3/mon:/etc/prometheus
                       prom/prometheus
3f0b8c2d1e9a7b6c5d4e3f2a1b0c9d8e7f6a5b4c3d2e1f0a9b8c7d6e5f4a3b2c
trainee@vault-lab:~$ sleep 15; curl -s localhost:9090/api/v1/targets | jq -r
                       '.data.activeTargets[] | "\(.labels.job) \(.health)"'
vault up
trainee@vault-lab:~$ curl -s 'localhost:9090/api/v1/query?query=vault_core_unsealed' | jq -r
                       '.data.result[] | "\(.metric.instance) \(.value[1])"'
127.0.0.1:8200 1
```

*Expected output — the vault target is up and the query returns 1*

✓ **Checkpoint:** target `vault up`. Open `http://localhost:9090`, graph `rate(vault_core_handle_request_count[1m])`, and run a few `vault kv get` commands to see it move. In production, Grafana dashboards and Alertmanager rules sit on top of this.

**Step 4 — Two audit devices.** Two devices, so one failing device cannot block Vault (study guide 6.3).

```
$ vault audit enable file file_path=$HOME/day3/mon/audit.log
$ vault audit enable -path=file2 file file_path=$HOME/day3/mon/audit2.log
```

**Step 5 — Generate activity as a named user.**

```
$ vault kv put secret/webapp db_user=webapp db_password=S3cure-v1
$ vault policy write webapp-read - <<< 'path "secret/data/webapp" { capabilities = ["read"]
}'
$ vault auth enable userpass
$ vault write auth/userpass/users/alice password=alice-pass policies=webapp-read
$ ALICE=$(vault login -method=userpass -token-only username=alice password=alice-pass)
$ VAULT_TOKEN=$ALICE vault kv get secret/webapp > /dev/null
```

**Step 6 — Trace it in the audit log.**

```
$ vault audit list
$ jq -c 'select(.type=="response" and .request.path=="secret/data/webapp")
         | {time, user: .auth.display_name, op: .request.operation}' ~/day3/mon/audit.log
$ jq -r 'select(.type=="response" and .auth.display_name=="userpass-alice") |
.response.data.data.db_password' ~/day3/mon/audit.log | tail -1
$ vault write -field=hash sys/audit-hash/file input="S3cure-v1"
```

Lab 3.6 · Step 6 · Trace a secret read in the audit log — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault audit list
Path      Type    Description
----      ----    -----------
file/     file    n/a
file2/    file    n/a
trainee@vault-lab:~$ jq -c 'select(.type=="response" and .request.path=="secret/data/webapp")
         | {time, user: .auth.display_name, op: .request.operation}' ~/day3/mon/audit.log
{"time":"2026-10-16T06:21:09.413Z","user":"root","op":"update"}
{"time":"2026-10-16T06:22:47.902Z","user":"userpass-alice","op":"read"}
trainee@vault-lab:~$ jq -r 'select(.type=="response" and .auth.display_name=="userpass-alice") |
                        .response.data.data.db_password' ~/day3/mon/audit.log | tail -1
hmac-sha256:7c1f0e6a5b0d4b7e2c9a8f3d1e6b5a4c3f2e1d0c9b8a7f6e5d4c3b2a1f0e9d8c
trainee@vault-lab:~$ vault write -field=hash sys/audit-hash/file input="S3cure-v1"
hmac-sha256:7c1f0e6a5b0d4b7e2c9a8f3d1e6b5a4c3f2e1d0c9b8a7f6e5d4c3b2a1f0e9d8c
```

*Expected output — two devices; alice's read is logged; the password appears only as an HMAC, which matches audit-hash*

✓ **Checkpoint:** the logged password is `hmac-sha256:…`, never the clear text, and the hash from `sys/audit-hash/file` is identical. Each device has its own HMAC key: hashing with `sys/audit-hash/file2` gives a different value that matches `audit2.log`.

**Step 7 — Health codes on a cluster.** If the Lab 3.5 cluster is running, compare an active node and a standby:

```
$ for p in 8210 8220 8230; do echo -n "$p "; curl -s -o /dev/null -w "%{http_code}\n"
http://127.0.0.1:$p/v1/sys/health; done
$ curl -s -o /dev/null -w "%{http_code}\n" "http://127.0.0.1:8220/v1/sys/health?
standbyok=true"
```

The active node returns 200 and standbys 429; with `standbyok=true` a standby returns 200 — the setting Kubernetes readiness probes use.

**Step 8 — Log rotation (read only).** In production the file device is rotated by `logrotate`, which must signal Vault to reopen the file:

```
# /etc/logrotate.d/vault
/var/log/vault/vault-audit.log {
  daily
  rotate 30
  compress
  delaycompress
  missingok
  notifempty
  postrotate
    /bin/systemctl reload vault 2>/dev/null || true
  endscript
}
```

## Clean up

```
$ docker rm -f prom
```

Keep the dev server for Lab 3.7.

# Lab 3.7 — Break-fix troubleshooting

**Objective:** diagnose and fix five common faults using only the error text, the server log, the audit log and Vault's own diagnostic commands, then collect a `vault debug` bundle. **Time:** 30 min. **Study guide:** Day 3, Module 7 (follow the decision tree: reach → sealed → token → policy → server). **Environment:** the dev server from Lab 3.6 (port 8200, audit devices enabled), a dev-TLS server on port 8300, and a throw-away server for fault 5.

For each fault: read the **whole** error, name the layer, then fix it. Try before reading the fix.

## Fault 1 — "It worked yesterday"

```
$ export VAULT_ADDR=https://127.0.0.1:8200
$ vault status
```

**Fix:** the error says the server answered in plain HTTP. The listener has TLS disabled, so the client must use `http://`.

```
$ export VAULT_ADDR=http://127.0.0.1:8200
$ vault status | grep Sealed
```

Lab 3.7 · Fault 1 · Wrong scheme — EXPECTED OUTPUT

```
trainee@vault-lab:~$ export VAULT_ADDR=https://127.0.0.1:8200
trainee@vault-lab:~$ vault status
Error checking seal status: Get "https://127.0.0.1:8200/v1/sys/seal-status": http: server
gave HTTP response to HTTPS client
trainee@vault-lab:~$ export VAULT_ADDR=http://127.0.0.1:8200
trainee@vault-lab:~$ vault status | grep Sealed
Sealed          false
```

*Expected output — HTTPS client against an HTTP listener; fixed by matching the scheme*

## Fault 2 — A server with a private CA

Start a TLS dev server in terminal 3. It creates its own CA and certificate in the folder you name.

```
$ mkdir -p ~/day3/devtls
$ vault server -dev -dev-tls -dev-tls-cert-dir=$HOME/day3/devtls -dev-listen-
address=127.0.0.1:8300 -dev-root-token-id=root
```

In terminal 2:

```
$ export VAULT_ADDR=https://127.0.0.1:8300
$ vault status
$ openssl s_client -connect 127.0.0.1:8300 </dev/null 2>/dev/null | grep -E
"^(subject|issuer)="
```

**Fix:** the certificate is signed by a CA the client does not trust. Point the client at that CA — never use `VAULT_SKIP_VERIFY` outside a lab.

```
$ export VAULT_CACERT=$HOME/day3/devtls/vault-ca.pem
```

```
$ vault status | grep -E "Sealed|Version"
```

Lab 3.7 · Fault 2 · Untrusted CA — EXPECTED OUTPUT

```
trainee@vault-lab:~$ export VAULT_ADDR=https://127.0.0.1:8300
trainee@vault-lab:~$ vault status
Error checking seal status: Get "https://127.0.0.1:8300/v1/sys/seal-status": tls: failed to
verify certificate: x509: certificate signed by unknown authority
trainee@vault-lab:~$ openssl s_client -connect 127.0.0.1:8300 </dev/null 2>/dev/null | grep -E
                       "^(subject|issuer)="
subject=CN = localhost
issuer=CN = Vault Dev CA
trainee@vault-lab:~$ export VAULT_CACERT=$HOME/day3/devtls/vault-ca.pem
trainee@vault-lab:~$ vault status | grep -E "Sealed|Version"
Sealed          false
Version         2.1.2
```

*Expected output — unknown authority; openssl shows the dev CA; trusting it fixes the call*

If the error were `certificate is valid for localhost, not vault.example.com`, the CA would be fine but the name you used is missing from the certificate's SANs — connect by a listed name or reissue the certificate. When done, press Ctrl+C in terminal 3 and run `unset VAULT_CACERT; export VAULT_ADDR=http://127.0.0.1:8200`.

## Fault 3 — Everything returns 503

Copy the `Unseal Key:` line from the dev server banner in terminal 1 into `UNSEAL_KEY` first.

```
$ export UNSEAL_KEY=<unseal key from the terminal 1 banner>
$ vault operator seal
$ vault kv get secret/webapp
$ curl -s -o /dev/null -w "%{http_code}\n" $VAULT_ADDR/v1/sys/health
```

**Fix:** 503 plus "Vault is sealed" is not a permissions problem. Check the server log (terminal 1), then unseal. With auto-unseal you would instead look for the KMS error in the log (Lab 3.3, Step 6).

```
$ vault operator unseal $UNSEAL_KEY | grep Sealed
```

Lab 3.7 · Fault 3 · Sealed server — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault operator seal
Success! Vault is sealed.
trainee@vault-lab:~$ vault kv get secret/webapp
Error making API request.

URL: GET http://127.0.0.1:8200/v1/sys/internal/ui/mounts/secret/webapp
Code: 503. Errors:

* Vault is sealed
trainee@vault-lab:~$ curl -s -o /dev/null -w "%{http_code}\n" $VAULT_ADDR/v1/sys/health
503
terminal 1 (server log): [INFO]  core: vault is sealed
trainee@vault-lab:~$ vault operator unseal $UNSEAL_KEY | grep Sealed
Sealed          false
```

*Expected output — sealed: 503 on every call; unsealing restores service*

## Fault 4 — "My policy allows it, but I get 403"

```
$ vault policy write webapp-broken - <<< 'path "secret/webapp" { capabilities = ["read"] }'
$ APP_TOKEN=$(vault token create -policy=webapp-broken -ttl=1h -field=token)
$ VAULT_TOKEN=$APP_TOKEN vault kv get secret/webapp
$ vault token capabilities $APP_TOKEN secret/data/webapp
$ jq -r 'select(.type=="response" and .error != null) | [.request.path, .error] | @tsv'
~/day3/mon/audit.log | tail -1
```

**Fix:** `vault kv get secret/webapp` really calls `secret/data/webapp` on a KV v2 mount (the URL in the error shows it). The policy must name the API path. Because tokens refer to policies by name, fixing the policy fixes existing tokens immediately.

```
$ vault policy write webapp-broken - <<< 'path "secret/data/webapp" { capabilities = ["read"]
}'
$ vault token capabilities $APP_TOKEN secret/data/webapp
$ VAULT_TOKEN=$APP_TOKEN vault kv get -field=db_user secret/webapp
```

Lab 3.7 · Fault 4 · KV v2 policy path — EXPECTED OUTPUT

```
trainee@vault-lab:~$ VAULT_TOKEN=$APP_TOKEN vault kv get secret/webapp
Error reading secret/data/webapp: Error making API request.

URL: GET http://127.0.0.1:8200/v1/secret/data/webapp
Code: 403. Errors:

* permission denied
trainee@vault-lab:~$ vault token capabilities $APP_TOKEN secret/data/webapp
deny
trainee@vault-lab:~$ vault policy read webapp-broken
path "secret/webapp" {
  capabilities = ["read"]
}
trainee@vault-lab:~$ vault policy write webapp-broken - <<< 'path "secret/data/webapp" {
                        capabilities = ["read"] }'
trainee@vault-lab:~$ vault token capabilities $APP_TOKEN secret/data/webapp
read
```

*Expected output — 403; capabilities deny on the data path; fixed policy gives read*

✓ **Checkpoint:** the audit log line shows `secret/data/webapp` with `permission denied` — in production this is how you find the exact path to put in the policy.

## Fault 5 — The server will not start

```
$ mkdir -p ~/day3/breakfix/data && chmod 000 ~/day3/breakfix/data
$ cat > ~/day3/breakfix/server.hcl <<EOF
disable_mlock = true
api_addr      = "http://127.0.0.1:8250"
cluster_addr  = "https://127.0.0.1:8251"
storage "raft" {
  path    = "$HOME/day3/breakfix/data"
  node_id = "breakfix"
}
listener "tcp" {
  address     = "127.0.0.1:8250"
  tls_disable = true
```

```
}
EOF
$ vault server -config=$HOME/day3/breakfix/server.hcl
```

**Fix:** the process exits at start-up, so there is no API to ask — the only evidence is the server's own output (`journalctl -u vault` for a service). It names the file and the reason. Give the Vault user ownership and access to its data directory, then start again (Ctrl+C after it starts):

```
$ ls -ld ~/day3/breakfix/data
$ chmod 700 ~/day3/breakfix/data
$ vault server -config=$HOME/day3/breakfix/server.hcl
```

Other start-up failures read the same way: `bind: address already in use` (another process holds the port), `Error parsing config` with a line number, `failed to lock memory` (add `IPC_LOCK` or set `disable_mlock = true` with Raft).

## Collect a debug bundle

On the dev server (port 8200, root token):

```
$ vault debug -duration=1m -interval=30s -output=$HOME/day3/vault-debug.tar.gz
$ tar tzf ~/day3/vault-debug.tar.gz | head
```

Lab 3.7 · Fault 5 and debug bundle — EXPECTED OUTPUT

```
trainee@vault-lab:~$ vault server -config=$HOME/day3/breakfix/server.hcl
Error initializing storage of type raft: failed to create fsm: failed to open bolt file: open
/home/trainee/day3/breakfix/data/vault.db: permission denied
trainee@vault-lab:~$ ls -ld ~/day3/breakfix/data
d--------- 2 trainee trainee 4096 Oct 16 12:04 /home/trainee/day3/breakfix/data
trainee@vault-lab:~$ chmod 700 ~/day3/breakfix/data
trainee@vault-lab:~$ vault debug -duration=1m -interval=30s
                     -output=$HOME/day3/vault-debug.tar.gz
==> Starting debug capture...
         Vault Address: http://127.0.0.1:8200
        Client Version: 2.1.2
        Server Version: 2.1.2
              Duration: 1m0s
              Interval: 30s
      Metrics Interval: 10s
               Targets: config, host, metrics, pprof, replication-status, server-status, log
                Output: /home/trainee/day3/vault-debug.tar.gz
...
Success! Bundle written to: /home/trainee/day3/vault-debug.tar.gz
```

*Expected output — the storage permission error, then a one-minute debug capture written to a tarball*

The bundle holds the sanitised config, host information, metrics, pprof profiles and server status — attach it to a HashiCorp support case, or open `metrics.json` and `pprof` files yourself during a performance incident.

## Extra faults to try

| Do this | What you will see | Layer |
|---|---|---|
| `VAULT_TOKEN=hvs.bogus vault kv get secret/webapp` | `403 permission denied` (Vault does not reveal whether the token exists) | Token |
| `vault token create -ttl=15s`, wait 20 s, use it | `403 permission denied`; `vault token lookup` on it fails | Token |
| Start a second dev server on 8200 | `listen tcp 127.0.0.1:8200: bind: address already in use` | Server |
| `vault server -dev -config=bad.hcl` with a missing `}` | `Error loading configuration ... At 7:1: ...` | Server |

## Clean up

```
$ rm -rf ~/day3/breakfix ~/day3/devtls
```

Keep `~/day3/vault-debug.tar.gz` if you want to explore it.

# Lab 3.8 — Terraform and Ansible automation

**Objective:** manage the Raft cluster's configuration with Terraform (including importing a mount that already exists), push a policy change through `plan` / `apply`, then run operational tasks — health check, snapshot, key rotation, secret write and certificate issue — with Ansible. **Time:** 40 min. **Study guide:** Day 3, Module 8. **Environment:** the Raft cluster from Lab 3.5, with Terraform and Ansible installed (How to use → One-time machine setup).

## Part A — Terraform

**Step 1 — Point at the cluster.** If you stopped it after Lab 3.5, start and unseal it first.

```
$ cd ~/day3 && ./raft-cluster.sh start          # only if it is stopped
$ KEY=$(jq -r '.unseal_keys_b64[0]' ~/day3/raft/init.json)
$ for p in 8210 8220 8230; do VAULT_ADDR=http://127.0.0.1:$p vault operator unseal $KEY >
/dev/null; done
$ export VAULT_ADDR=http://127.0.0.1:8210
$ export VAULT_TOKEN=$(jq -r .root_token ~/day3/raft/init.json)
$ mkdir -p ~/day3/tf/policies && cd ~/day3/tf
```

The provider reads `VAULT_ADDR` and `VAULT_TOKEN`. Using the root token is a lab shortcut; the study guide shows the provider logging in with its own AppRole.

**Step 2 — Write the configuration.**

```
# versions.tf
terraform {
  required_version = ">= 1.6"
  required_providers {
    vault = {
      source  = "hashicorp/vault"
      version = "~> 5.0"
    }
  }
  # Lab: local state. Production: an encrypted remote backend (study guide 8.1).
}

provider "vault" {}
```

```
# main.tf
# secret/ already exists (Lab 3.5): adopt it instead of recreating it
import {
  to = vault_mount.kv
  id = "secret"
}

resource "vault_mount" "kv" {
  path    = "secret"
  type    = "kv"
  options = { version = "2" }
}

resource "vault_policy" "webapp" {
  name   = "webapp"
  policy = file("${path.module}/policies/webapp.hcl")
}
```

```
resource "vault_policy" "ops" {
  name   = "ops"
  policy = file("${path.module}/policies/ops.hcl")
}

resource "vault_auth_backend" "approle" {
  type = "approle"
}

resource "vault_approle_auth_backend_role" "webapp" {
  backend        = vault_auth_backend.approle.path
  role_name      = "webapp"
  token_policies = [vault_policy.webapp.name]
  token_ttl      = 3600
  token_max_ttl  = 14400
}

resource "vault_approle_auth_backend_role" "ops" {
  backend        = vault_auth_backend.approle.path
  role_name      = "ops"
  token_policies = [vault_policy.ops.name]
  token_ttl      = 900
}

resource "vault_mount" "transit" {
  path = "transit"
  type = "transit"
}

resource "vault_transit_secret_backend_key" "orders" {
  backend          = vault_mount.transit.path
  name             = "orders"
  type             = "aes256-gcm96"
  deletion_allowed = false
}

resource "vault_mount" "pki" {
  path                  = "pki"
  type                  = "pki"
  max_lease_ttl_seconds = 315360000
}

resource "vault_pki_secret_backend_root_cert" "root" {
  backend     = vault_mount.pki.path
  type        = "internal"
  common_name = "StackMaster Root CA"
  ttl         = "87600h"
  issuer_name = "root-2026"
}

resource "vault_pki_secret_backend_role" "web" {
  backend          = vault_mount.pki.path
  name             = "web-server"
  allowed_domains  = ["stackmaster.lab"]
  allow_subdomains = true
  max_ttl          = "720h"
  key_type         = "rsa"
  key_bits         = 2048
}

resource "vault_audit" "file" {
  type    = "file"
```

```
  options = { file_path = pathexpand("~/day3/raft/tf-audit.log") }
}
```

```
# policies/webapp.hcl
path "secret/data/webapp" {
  capabilities = ["read"]
}
```

```
# policies/ops.hcl — what the Ansible playbooks need, nothing more
path "sys/storage/raft/snapshot" {
  capabilities = ["read"]
}
path "transit/keys/orders/rotate" {
  capabilities = ["update"]
}
path "secret/data/webapp" {
  capabilities = ["create", "update", "read"]
}
path "pki/issue/web-server" {
  capabilities = ["update"]
}
```

To keep the lab short, certificates come straight from the root mount here. Production issues from an intermediate (Lab 3.1); the provider has `vault_pki_secret_backend_intermediate_cert_request`, `..._root_sign_intermediate` and `..._intermediate_set_signed` for exactly those steps.

**Step 3 — Init and plan.**

```
$ terraform init -input=false | grep -E "Installing|initialized"
$ terraform plan -out=day3.tfplan | grep -E "^  # |Plan:"
```

Lab 3.8 · Step 3 · terraform plan — EXPECTED OUTPUT

```
trainee@vault-lab:~$ terraform init -input=false | grep -E "Installing|initialized"
- Installing hashicorp/vault v5.3.0...
Terraform has been successfully initialized!
trainee@vault-lab:~$ terraform plan -out=day3.tfplan | grep -E "^  # |Plan:"
  # vault_approle_auth_backend_role.ops will be created
  # vault_approle_auth_backend_role.webapp will be created
  # vault_audit.file will be created
  # vault_auth_backend.approle will be created
  # vault_mount.kv will be imported
  # vault_mount.pki will be created
  # vault_mount.transit will be created
  # vault_pki_secret_backend_role.web will be created
  # vault_pki_secret_backend_root_cert.root will be created
  # vault_policy.ops will be created
  # vault_policy.webapp will be created
  # vault_transit_secret_backend_key.orders will be created
Plan: 1 to import, 11 to add, 0 to change, 0 to destroy.
```

*Expected output — provider installed; plan imports secret/ and adds eleven resources*

**Step 4 — Apply and check in Vault.**

```
$ terraform apply day3.tfplan | tail -1
$ vault secrets list | grep -E "^(secret|pki|transit)/"
```

```
$ vault policy list
$ vault read -field=token_policies auth/approle/role/webapp
```

Lab 3.8 · Step 4 · Apply and verify in Vault — EXPECTED OUTPUT

```
trainee@vault-lab:~$ terraform apply day3.tfplan | tail -1
Apply complete! Resources: 1 imported, 11 added, 0 changed, 0 destroyed.
trainee@vault-lab:~$ vault secrets list | grep -E "^(secret|pki|transit)/"
pki/          pki          pki_7c0e2b1a          n/a
secret/       kv           kv_3d9f6e10           n/a
transit/      transit      transit_a41c8b25      n/a
trainee@vault-lab:~$ vault policy list
default
ops
webapp
root
trainee@vault-lab:~$ vault read -field=token_policies auth/approle/role/webapp
[webapp]
```

*Expected output — apply complete; mounts, policies and the AppRole role exist in Vault*

✓ **Checkpoint:** `terraform plan` now prints `No changes. Your infrastructure matches the configuration.` and `vault kv get secret/cluster-test` still works — the import kept the existing data.

**Step 5 — A change through review.** The webapp now also needs to encrypt orders. Add to `policies/webapp.hcl`:

```
path "transit/encrypt/orders" {
  capabilities = ["update"]
}
```

```
$ terraform plan | sed -n '/# vault_policy.webapp/,/^    }/p'
$ terraform apply -auto-approve | tail -1
```

Lab 3.8 · Step 5 · A policy change through plan — EXPECTED OUTPUT

```
trainee@vault-lab:~$ terraform plan | sed -n '/# vault_policy.webapp/,/^    }/p'
  # vault_policy.webapp will be updated in-place
  ~ resource "vault_policy" "webapp" {
        id     = "webapp"
        name   = "webapp"
      ~ policy = <<-EOT
            path "secret/data/webapp" {
              capabilities = ["read"]
            }
          + path "transit/encrypt/orders" {
          +   capabilities = ["update"]
          + }
          EOT
    }
Plan: 0 to add, 1 to change, 0 to destroy.
```

*Expected output — an in-place update that adds exactly the new path*

In a team this diff is what reviewers approve in the pull request, and CI runs the apply. Try the opposite too: change the policy by hand with `vault policy write webapp -` and run `terraform plan` — Terraform reports the drift and offers to put it back.

## Part B — Ansible

**Step 6 — Credentials for the ops AppRole.** Terraform created the role; fetch its IDs.

```
$ mkdir -p ~/day3/ansible/backups && cd ~/day3/ansible
$ vault read -field=role_id auth/approle/role/ops/role-id > ops_role_id
$ vault write -f -field=secret_id auth/approle/role/ops/secret-id > ops_secret_id
$ chmod 600 ops_role_id ops_secret_id
```

**Step 7 — The operations playbook.** The same tasks as the study guide (8.2), adjusted for the lab cluster (HTTP, local paths).

```
# vault-ops.yml
- name: Vault operational tasks
  hosts: localhost
  connection: local
  gather_facts: true
  vars:
    vault_addr: http://127.0.0.1:8210
    ops_role_id: "{{ lookup('file', 'ops_role_id') }}"
    ops_secret_id: "{{ lookup('file', 'ops_secret_id') }}"
    backup_dir: "{{ lookup('env', 'HOME') }}/day3/ansible/backups"
  module_defaults:
    group/community.hashi_vault.vault:
      url: "{{ vault_addr }}"
      auth_method: approle
      role_id: "{{ ops_role_id }}"
      secret_id: "{{ ops_secret_id }}"

  tasks:
    - name: Health check (200 = active, 429 = standby)
      ansible.builtin.uri:
        url: "{{ vault_addr }}/v1/sys/health"
        status_code: [200, 429]

    - name: Log in once and reuse the token
      community.hashi_vault.vault_login:
      register: login

    - name: Take a Raft snapshot
      ansible.builtin.uri:
        url: "{{ vault_addr }}/v1/sys/storage/raft/snapshot"
        headers:
          X-Vault-Token: "{{ login.login.auth.client_token }}"
        dest: "{{ backup_dir }}/vault-{{ ansible_date_time.iso8601_basic_short }}.snap"
      no_log: true

    - name: Rotate the Transit key for orders
      community.hashi_vault.vault_write:
        path: transit/keys/orders/rotate

    - name: Write a KV v2 secret
      community.hashi_vault.vault_kv2_write:
        engine_mount_point: secret
        path: webapp
        data:
          db_user: webapp
          db_password: "{{ new_db_password }}"
      no_log: true
```

**Step 8 — Run it.**

```
$ ansible-playbook vault-ops.yml -e new_db_password=Ansible-v1
$ vault read -field=latest_version transit/keys/orders
$ ls -lh backups/
$ vault kv get -field=db_password secret/webapp
```

Lab 3.8 · Step 8 · ansible-playbook vault-ops.yml — EXPECTED OUTPUT

```
trainee@vault-lab:~$ ansible-playbook vault-ops.yml

PLAY [Vault operational tasks] *************************************************

TASK [Health check (200 = active, 429 = standby)] ******************************
ok: [localhost]
TASK [Log in once and reuse the token] *****************************************
ok: [localhost]
TASK [Take a Raft snapshot] ****************************************************
changed: [localhost]
TASK [Rotate the Transit key for orders] ***************************************
changed: [localhost]
TASK [Write a KV v2 secret] ****************************************************
changed: [localhost]

PLAY RECAP *********************************************************************
localhost   : ok=5    changed=3    unreachable=0    failed=0    skipped=0
trainee@vault-lab:~$ vault read -field=latest_version transit/keys/orders
2
```

*Expected output — five tasks, three changes; the orders key moved to version 2*

✓ **Checkpoint:** a `.snap` file in `backups/`, `latest_version 2`, and the secret reads `Ansible-v1`. Run the playbook again: the KV write reports `ok` instead of `changed`, because the data is already there — the module is idempotent. The rotation is not (each run adds a version), which is why rotations belong on a schedule rather than in every deployment.

**Step 9 — Issue a certificate with Ansible.**

```
# issue-cert.yml
- name: Issue a TLS certificate from Vault PKI
  hosts: localhost
  connection: local
  gather_facts: false
  vars:
    out_dir: "{{ lookup('env', 'HOME') }}/day3/ansible/tls"
  module_defaults:
    group/community.hashi_vault.vault:
      url: http://127.0.0.1:8210
      auth_method: approle
      role_id: "{{ lookup('file', 'ops_role_id') }}"
      secret_id: "{{ lookup('file', 'ops_secret_id') }}"
  tasks:
    - name: Output folder
      ansible.builtin.file:
        path: "{{ out_dir }}"
        state: directory
        mode: "0700"

    - name: Issue the certificate
```

```
      community.hashi_vault.vault_pki_generate_certificate:
        engine_mount_point: pki
        role_name: web-server
        common_name: app1.stackmaster.lab
        ttl: 24h
      register: cert
      no_log: true

    - name: Write certificate and key
      ansible.builtin.copy:
        content: "{{ item.content }}"
        dest: "{{ out_dir }}/{{ item.name }}"
        mode: "{{ item.mode }}"
      loop:
        - { name: app1.crt, content: "{{ cert.data.data.certificate }}", mode: "0644" }
        - { name: app1.key, content: "{{ cert.data.data.private_key }}", mode: "0600" }
      loop_control:
        label: "{{ item.name }}"
      no_log: true
```

```
$ ansible-playbook issue-cert.yml
$ openssl x509 -in ~/day3/ansible/tls/app1.crt -noout -subject -issuer -enddate
```

✓ **Checkpoint:** subject `CN = app1.stackmaster.lab`, issuer `CN = StackMaster Root CA`, valid for 24 hours. For a fleet, change `hosts:` to your web group, delegate the issue task to `localhost`, and copy the files to each host (study guide 8.2).

**Step 10 — Schedule it (discussion).** In production the ops playbook runs from AWX / Ansible Automation Platform or a CI scheduler with its own AppRole, nightly for snapshots and monthly for rotations, and the snapshot files are copied to object storage.

## Clean up

```
$ cd ~/day3/tf && terraform destroy -auto-approve      # optional: removes what Terraform
created
$ cd ~/day3 && ./raft-cluster.sh destroy
$ export VAULT_ADDR=http://127.0.0.1:8200 VAULT_TOKEN=root
```

`terraform destroy` would also remove the imported `secret/` mount and its data — Terraform now owns it.

# Lab 3.9 — Vault HA on Kubernetes with Helm and VSO

**Objective:** deploy a three-replica Raft Vault cluster on Kubernetes with the official Helm chart, initialise and join it, configure Kubernetes auth, deliver a secret to one app through the Agent injector and to another through the Vault Secrets Operator, and watch a secret change roll out. **Time:** 45 min. **Study guide:** Day 3, Module 9 (Day 2 Lab 2.5 used a single dev-mode pod; this lab is the production shape). **Environment:** Docker, kind, kubectl, helm. The lab uses plain HTTP and Shamir keys inside the cluster; study guide 9.2–9.3 shows the TLS and KMS auto-unseal values for a real cluster.

## Steps

**Step 1 — A cluster and the values file.**

```
$ kind create cluster --name vault-ha
$ mkdir -p ~/day3/k8s && cd ~/day3/k8s
```

```
# vault-ha-kind.yaml
server:
  image:
    tag: "2.1.2"
  affinity: ""            # kind has one node; production keeps the chart's one-pod-per-node
rule
  dataStorage:
    size: 1Gi
  ha:
    enabled: true
    replicas: 3
    raft:
      enabled: true
      setNodeId: true
      config: |
        ui = true
        disable_mlock = true
        listener "tcp" {
          tls_disable     = 1
          address         = "[::]:8200"
          cluster_address = "[::]:8201"
        }
        storage "raft" {
          path = "/vault/data"
          retry_join { leader_api_addr = "http://vault-0.vault-internal:8200" }
          retry_join { leader_api_addr = "http://vault-1.vault-internal:8200" }
          retry_join { leader_api_addr = "http://vault-2.vault-internal:8200" }
        }
        service_registration "kubernetes" {}
injector:
  enabled: true
```

**Step 2 — Install.**

```
$ helm repo add hashicorp https://helm.releases.hashicorp.com && helm repo update
$ helm install vault hashicorp/vault -n vault --create-namespace -f vault-ha-kind.yaml
$ kubectl -n vault get pods
```

Lab 3.9 · Step 2 · helm install (HA Raft) EXPECTED OUTPUT

```
trainee@vault-lab:~$ helm install vault hashicorp/vault -n vault --create-namespace -f
                     vault-ha-kind.yaml
NAME: vault
LAST DEPLOYED: Fri Oct 16 12:31:05 2026
NAMESPACE: vault
STATUS: deployed
REVISION: 1
trainee@vault-lab:~$ kubectl -n vault get pods
NAME                                    READY   STATUS    RESTARTS   AGE
vault-0                                 0/1     Running   0          48s
vault-1                                 0/1     Running   0          48s
vault-2                                 0/1     Running   0          48s
vault-agent-injector-6b8f9c7d5-x2kqp    1/1     Running   0          48s
0/1 is expected: the readiness probe fails until each pod is initialised and unsealed.
```

*Expected output — three Vault pods Running but not Ready, and the injector Ready*

**Step 3 — Initialise vault-0, unseal all three.** vault-1 and vault-2 find vault-0 through `retry_join` and the `vault-internal` headless service.

```
$ kubectl -n vault exec vault-0 -- vault operator init -key-shares=1 -key-threshold=1 -
format=json > ~/day3/k8s/init.json
$ KEY=$(jq -r '.unseal_keys_b64[0]' ~/day3/k8s/init.json); ROOT=$(jq -r .root_token
~/day3/k8s/init.json)
$ for p in vault-0 vault-1 vault-2; do kubectl -n vault exec $p -- vault operator unseal $KEY
> /dev/null; sleep 5; done
$ kubectl -n vault exec vault-0 -- sh -c "VAULT_TOKEN=$ROOT vault operator raft list-peers"
$ kubectl -n vault get pods -l app.kubernetes.io/name=vault -L vault-active
```

Lab 3.9 · Step 3 · Initialise, unseal, join EXPECTED OUTPUT

```
trainee@vault-lab:~$ kubectl -n vault exec vault-0 -- vault operator init -key-shares=1
                     -key-threshold=1 -format=json > ~/day3/k8s/init.json
trainee@vault-lab:~$ KEY=$(jq -r '.unseal_keys_b64[0]' ~/day3/k8s/init.json); ROOT=$(jq -r
                     .root_token ~/day3/k8s/init.json)
trainee@vault-lab:~$ for p in vault-0 vault-1 vault-2; do kubectl -n vault exec $p -- vault
                     operator unseal $KEY > /dev/null; sleep 5; done
trainee@vault-lab:~$ kubectl -n vault exec vault-0 -- sh -c "VAULT_TOKEN=$ROOT vault operator
                     raft list-peers"
Node       Address                         State       Voter
----       -------                         -----       -----
vault-0    vault-0.vault-internal:8201     leader      true
vault-1    vault-1.vault-internal:8201     follower    true
vault-2    vault-2.vault-internal:8201     follower    true
trainee@vault-lab:~$ kubectl -n vault get pods -l app.kubernetes.io/name=vault -L vault-active
NAME      READY   STATUS    RESTARTS   AGE     VAULT-ACTIVE
vault-0   1/1     Running   0          4m12s   true
vault-1   1/1     Running   0          4m12s   false
vault-2   1/1     Running   0          4m12s   false
```

*Expected output — three Raft voters; all pods 1/1 Ready; vault-0 labelled active*

✓ **Checkpoint:** three voters and `VAULT-ACTIVE true` on exactly one pod. That label (from `service_registration "kubernetes"`) is what the `vault-active` Service selects.

**Step 4 — Configure Vault from inside the active pod.**

```
$ kubectl -n vault exec -it vault-0 -- sh
/ $ export VAULT_TOKEN=<root token from init.json>
/ $ vault secrets enable -path=secret kv-v2
/ $ vault kv put secret/webapp db_user=webapp db_password=K8s-pass-v1
/ $ vault policy write webapp - <<EOF
path "secret/data/webapp" { capabilities = ["read"] }
EOF
/ $ vault auth enable kubernetes
/ $ vault write auth/kubernetes/config kubernetes_host="https://kubernetes.default.svc:443"
/ $ vault write auth/kubernetes/role/webapp-agent \
      bound_service_account_names=webapp bound_service_account_namespaces=webapp \
      audience="https://kubernetes.default.svc.cluster.local" token_policies=webapp
token_ttl=1h
/ $ vault write auth/kubernetes/role/webapp-vso \
      bound_service_account_names=webapp bound_service_account_namespaces=webapp \
      audience=vault token_policies=webapp token_ttl=1h
/ $ exit
```

Two roles, because the two clients present different tokens: the injected Agent uses the pod's normal ServiceAccount token, whose audience is the API server (`https://kubernetes.default.svc.cluster.local` on kind), while VSO requests a token with the audience you choose (`vault`). Binding the audience stops a token meant for one service being replayed to another.

**Step 5 — App 1: the Agent injector.**

```
$ kubectl create namespace webapp
$ kubectl -n webapp create serviceaccount webapp
```

```
# webapp-agent.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: webapp-agent
  namespace: webapp
spec:
  replicas: 1
  selector:
    matchLabels: { app: webapp-agent }
  template:
    metadata:
      labels: { app: webapp-agent }
      annotations:
        vault.hashicorp.com/agent-inject: "true"
        vault.hashicorp.com/role: "webapp-agent"
        vault.hashicorp.com/agent-inject-secret-db.env: "secret/data/webapp"
        vault.hashicorp.com/agent-inject-template-db.env: |
          {{- with secret "secret/data/webapp" -}}
          DB_USER={{ .Data.data.db_user }}
          DB_PASSWORD={{ .Data.data.db_password }}
          {{- end }}
    spec:
      serviceAccountName: webapp
      containers:
        - name: app
          image: busybox:1.36
          command: ["sh", "-c", "while true; do cat /vault/secrets/db.env; sleep 30; done"]
```

```
$ kubectl apply -f webapp-agent.yaml
$ kubectl -n webapp get pods
$ kubectl -n webapp get pod -l app=webapp-agent -o
jsonpath='{.items[0].spec.containers[*].name}'; echo
$ kubectl -n webapp logs deploy/webapp-agent -c app --tail=2
```

Lab 3.9 · Step 5 · Injected Agent sidecar — EXPECTED OUTPUT

```
trainee@vault-lab:~$ kubectl -n webapp get pods
NAME                             READY   STATUS    RESTARTS   AGE
webapp-agent-7d9c8b6f5-kq4mz     2/2     Running   0          35s
trainee@vault-lab:~$ kubectl -n webapp get pod -l app=webapp-agent -o
                     jsonpath='{.items[0].spec.containers[*].name}'; echo
app vault-agent
trainee@vault-lab:~$ kubectl -n webapp logs deploy/webapp-agent -c app --tail=2
DB_USER=webapp
DB_PASSWORD=K8s-pass-v1
```

*Expected output — 2/2 containers (app + vault-agent); the app reads the rendered file*

### Step 6 — App 2: the Vault Secrets Operator.

```
$ helm install vault-secrets-operator hashicorp/vault-secrets-operator \
    -n vault-secrets-operator-system --create-namespace
$ kubectl -n vault-secrets-operator-system get pods
```

```
# webapp-vso.yaml
apiVersion: secrets.hashicorp.com/v1beta1
kind: VaultConnection
metadata: { name: vault, namespace: webapp }
spec:
  address: http://vault-active.vault.svc:8200
---
apiVersion: secrets.hashicorp.com/v1beta1
kind: VaultAuth
metadata: { name: webapp, namespace: webapp }
spec:
  vaultConnectionRef: vault
  method: kubernetes
  mount: kubernetes
  kubernetes:
    role: webapp-vso
    serviceAccount: webapp
    audiences: ["vault"]
---
apiVersion: secrets.hashicorp.com/v1beta1
kind: VaultStaticSecret
metadata: { name: webapp-db, namespace: webapp }
spec:
  vaultAuthRef: webapp
  mount: secret
  type: kv-v2
  path: webapp
  refreshAfter: 30s
  destination:
    name: webapp-db
    create: true
  rolloutRestartTargets:
    - kind: Deployment
```

```
      name: webapp-vso
---
apiVersion: apps/v1
kind: Deployment
metadata: { name: webapp-vso, namespace: webapp }
spec:
  replicas: 1
  selector:
    matchLabels: { app: webapp-vso }
  template:
    metadata:
      labels: { app: webapp-vso }
    spec:
      containers:
        - name: app
          image: busybox:1.36
          command: ["sh", "-c", "echo DB_PASSWORD=$DB_PASSWORD; sleep 3600"]
          env:
            - name: DB_PASSWORD
              valueFrom:
                secretKeyRef: { name: webapp-db, key: db_password }
```

```
$ kubectl apply -f webapp-vso.yaml
$ kubectl -n webapp get vaultstaticsecret webapp-db
$ kubectl -n webapp get secret webapp-db -o jsonpath='{.data.db_password}' | base64 -d; echo
```

**Step 7 — Rotate the secret and watch both apps.**

```
$ kubectl -n vault exec vault-0 -- sh -c "VAULT_TOKEN=$ROOT vault kv put secret/webapp
db_user=webapp db_password=K8s-pass-v2" > /dev/null
$ sleep 40; kubectl -n webapp get secret webapp-db -o jsonpath='{.data.db_password}' | base64
-d; echo
$ kubectl -n webapp get pods -l app=webapp-vso
$ kubectl -n webapp logs deploy/webapp-vso
```

Lab 3.9 · Step 7 · VSO sync and rollout restart — EXPECTED OUTPUT

```
trainee@vault-lab:~$ kubectl -n webapp get vaultstaticsecret webapp-db
NAME        AGE
webapp-db   52s
trainee@vault-lab:~$ kubectl -n webapp get secret webapp-db -o jsonpath='{.data.db_password}' |
                     base64 -d; echo
K8s-pass-v1
trainee@vault-lab:~$ kubectl -n vault exec vault-0 -- sh -c "VAULT_TOKEN=$ROOT vault kv put
                     secret/webapp db_user=webapp db_password=K8s-pass-v2" > /dev/null
trainee@vault-lab:~$ sleep 40; kubectl -n webapp get secret webapp-db -o
                     jsonpath='{.data.db_password}' | base64 -d; echo
K8s-pass-v2
trainee@vault-lab:~$ kubectl -n webapp get pods -l app=webapp-vso
NAME                          READY   STATUS        RESTARTS   AGE
webapp-vso-5f7b9d8c4-7mxrt    1/1     Terminating   0          3m1s
webapp-vso-6c4d2f9b7-p8wzn    1/1     Running       0          6s
```

*Expected output — VSO updates the Kubernetes Secret and restarts the Deployment*

✓ **Checkpoint:** the Secret holds `K8s-pass-v2`, a new `webapp-vso` pod started a few seconds ago and logs `DB_PASSWORD=K8s-pass-v2`. The injected sidecar in `webapp-agent` also re-renders `db.env` — within five minutes by default, the Agent's static-secret interval.

**Step 8 — Lose the leader.**

```
$ kubectl -n vault delete pod vault-0
$ sleep 15; kubectl -n vault get pods -l app.kubernetes.io/name=vault -L vault-active,vault-
sealed
$ kubectl -n vault exec vault-0 -- vault operator unseal $KEY > /dev/null
$ kubectl -n webapp get secret webapp-db -o jsonpath='{.data.db_password}' | base64 -d; echo
```

vault-1 or vault-2 becomes active and the apps keep working. The recreated vault-0 comes back **sealed** — with Shamir keys someone must unseal every restarted pod, which is why production Kubernetes deployments use KMS auto-unseal (study guide 9.3).

## Clean up

```
$ kind delete cluster --name vault-ha
```

# Lab 3.10 — Capstone: PKI + Transit + Vault Agent

**Objective:** build **StackMaster Orders** end to end: nginx serves HTTPS with a 24-hour certificate from Vault PKI that Vault Agent renews and reloads; an orders app stores card numbers encrypted by Transit through the Agent's API proxy, holding no Vault token; everything is visible in the audit log. **Time:** 45 min. **Study guide:** Day 3, Module 10 (architecture diagram and success criteria). **Environment:** a fresh dev server, nginx, Python 3 with `requests`, sqlite3.

## Part A — The Vault side

**Step 1 — Fresh dev server and folders.** Press Ctrl+C on the old dev server and start a new one in terminal 1 (`vault server -dev -dev-root-token-id=root`). In terminal 2:

```
$ export VAULT_ADDR=http://127.0.0.1:8200 VAULT_TOKEN=root
$ sudo mkdir -p /opt/capstone/tls /opt/capstone/tmp && sudo chown -R $USER: /opt/capstone
$ cd /opt/capstone
$ echo "127.0.0.1 orders.stackmaster.lab" | sudo tee -a /etc/hosts
```

**Step 2 — The policy.** Exactly what the capstone needs (study guide 10.2):

```
# /opt/capstone/capstone.hcl
path "pki_int/issue/web-server" {
  capabilities = ["update"]
}
path "transit/encrypt/orders" {
  capabilities = ["update"]
}
path "transit/decrypt/orders" {
  capabilities = ["update"]
}
path "transit/rewrap/orders" {
  capabilities = ["update"]
}
```

**Step 3 — Build PKI, Transit, AppRole and audit in one script.** It condenses Labs 3.1 and 3.2.

```
#!/usr/bin/env bash
# /opt/capstone/capstone-vault.sh — Vault side of the capstone (dev server, root token)
set -euo pipefail
cd /opt/capstone

# Two-tier PKI
vault secrets enable pki
vault secrets tune -max-lease-ttl=87600h pki > /dev/null
vault write -field=certificate pki/root/generate/internal \
  common_name="StackMaster Root CA" issuer_name=root-2026 ttl=87600h > tls/root_ca.pem
vault secrets enable -path=pki_int pki
vault secrets tune -max-lease-ttl=43800h pki_int > /dev/null
vault write -field=csr pki_int/intermediate/generate/internal \
  common_name="StackMaster Intermediate CA" issuer_name=int-2026 > pki_int.csr
vault write -field=certificate pki/root/sign-intermediate issuer_ref=root-2026 \
  csr=@pki_int.csr format=pem_bundle ttl=43800h > int.pem
vault write pki_int/intermediate/set-signed certificate=@int.pem > /dev/null
echo "intermediate imported: StackMaster Intermediate CA"
vault write pki_int/roles/web-server allowed_domains=stackmaster.lab \
  allow_subdomains=true max_ttl=24h > /dev/null
```

```
# Transit
vault secrets enable transit
vault write -f transit/keys/orders > /dev/null

# Policy, AppRole, audit
vault policy write capstone capstone.hcl
vault auth enable approle
vault audit enable file file_path=/opt/capstone/audit.log
vault write auth/approle/role/capstone token_policies=capstone token_ttl=1h token_max_ttl=4h
> /dev/null
vault read -field=role_id auth/approle/role/capstone/role-id > role_id
vault write -f -field=secret_id auth/approle/role/capstone/secret-id > secret_id
chmod 600 role_id secret_id
echo "role_id and secret_id written to /opt/capstone (mode 0600)"
```

```
$ chmod +x capstone-vault.sh && ./capstone-vault.sh
$ vault token capabilities $(vault write -field=token auth/approle/login role_id=$(cat
/opt/capstone/role_id) secret_id=$(cat /opt/capstone/secret_id)) transit/keys/orders
```

Lab 3.10 · Step 3 · Capstone Vault side — EXPECTED OUTPUT

```
trainee@vault-lab:~$ ./capstone-vault.sh
Success! Enabled the pki secrets engine at: pki/
Success! Enabled the pki secrets engine at: pki_int/
intermediate imported: StackMaster Intermediate CA
Success! Enabled the transit secrets engine at: transit/
Success! Uploaded policy: capstone
Success! Enabled approle auth method at: approle/
Success! Enabled the file audit device at: file/
role_id and secret_id written to /opt/capstone (mode 0600)
trainee@vault-lab:~$ vault token capabilities $(vault write -field=token auth/approle/login
                       role_id=$(cat /opt/capstone/role_id) secret_id=$(cat
                       /opt/capstone/secret_id)) transit/keys/orders
deny
```

*Expected output — PKI, Transit, policy, AppRole and audit in place; the capstone identity cannot manage the key*

## Part B — Vault Agent

### Step 4 — The certificate template.

```
$ cat > /opt/capstone/web.pem.ctmpl <<'EOF'
{{- with pkiCert "pki_int/issue/web-server" "common_name=orders.stackmaster.lab" "ttl=24h"
-}}
{{ .Cert }}{{ .CA }}{{ .Key }}
{{ .Key | writeToFile "/opt/capstone/tls/web.key" "" "" "0600" }}
{{ print .Cert "\n" .CA | writeToFile "/opt/capstone/tls/web.crt" "" "" "0644" }}
{{ .CA | writeToFile "/opt/capstone/tls/ca.crt" "" "" "0644" }}
{{- end -}}
EOF
```

**Step 5 — The Agent configuration.** The lab version of study guide 10.3 (plain HTTP to the dev server).

```
# /opt/capstone/agent-capstone.hcl
pid_file = "/opt/capstone/agent.pid"

vault {
  address = "http://127.0.0.1:8200"
```

```
}

auto_auth {
  method "approle" {
    config = {
      role_id_file_path                   = "/opt/capstone/role_id"
      secret_id_file_path                 = "/opt/capstone/secret_id"
      remove_secret_id_file_after_reading = false
    }
  }
  sink "file" {
    config = {
      path = "/opt/capstone/agent-token"
      mode = 0600
    }
  }
}

cache {}
api_proxy {
  use_auto_auth_token = "force"
}
listener "tcp" {
  address     = "127.0.0.1:8100"
  tls_disable = true
}

template {
  source      = "/opt/capstone/web.pem.ctmpl"
  destination = "/opt/capstone/tls/web.pem"
  perms       = "0600"
  exec {
    command = ["sh", "-c", "nginx -p /opt/capstone -c nginx.conf -s reload || true"]
  }
}
```

Start it in terminal 3 and leave it running:

```
$ cd /opt/capstone && vault agent -config=agent-capstone.hcl
```

The first render runs the reload command before nginx is running; `|| true` keeps that harmless.

## Part C — nginx with the Vault certificate

### Step 6 — An nginx that runs as you, on port 8443.

```
# /opt/capstone/nginx.conf
pid        /opt/capstone/nginx.pid;
error_log  /opt/capstone/nginx-error.log;
events {}
http {
  access_log            /opt/capstone/nginx-access.log;
  client_body_temp_path /opt/capstone/tmp/body;
  proxy_temp_path       /opt/capstone/tmp/proxy;
  fastcgi_temp_path     /opt/capstone/tmp/fastcgi;
  uwsgi_temp_path       /opt/capstone/tmp/uwsgi;
  scgi_temp_path        /opt/capstone/tmp/scgi;

  server {
    listen 8443 ssl;
```

```
    server_name orders.stackmaster.lab;
    ssl_certificate     /opt/capstone/tls/web.crt;
    ssl_certificate_key /opt/capstone/tls/web.key;
    location / {
      default_type text/plain;
      return 200 "StackMaster Orders - TLS by Vault PKI\n";
    }
  }
}
```

**Step 7 — Start nginx and test the chain.** In terminal 2:

```
$ ls /opt/capstone/tls
$ nginx -p /opt/capstone -c nginx.conf
$ curl --cacert /opt/capstone/tls/root_ca.pem https://orders.stackmaster.lab:8443
$ openssl s_client -connect orders.stackmaster.lab:8443 -CAfile /opt/capstone/tls/root_ca.pem
</dev/null 2>/dev/null | grep -E "^(subject|issuer)=|Verify return code"
```

Lab 3.10 · Step 7 · HTTPS with a Vault PKI certificate — EXPECTED OUTPUT

```
trainee@vault-lab:~$ ls /opt/capstone/tls
ca.crt  root_ca.pem  web.crt  web.key  web.pem
trainee@vault-lab:~$ nginx -p /opt/capstone -c nginx.conf
trainee@vault-lab:~$ curl --cacert /opt/capstone/tls/root_ca.pem
                         https://orders.stackmaster.lab:8443
StackMaster Orders - TLS by Vault PKI
trainee@vault-lab:~$ openssl s_client -connect orders.stackmaster.lab:8443 -CAfile
                         /opt/capstone/tls/root_ca.pem </dev/null 2>/dev/null \
    | grep -E "^(subject|issuer)=|Verify return code"
subject=CN = orders.stackmaster.lab
issuer=CN = StackMaster Intermediate CA
Verify return code: 0 (ok)
```

*Expected output — HTTPS works with only the root CA trusted; issuer is the intermediate; verify OK*

nginx may print one alert that it cannot open `/var/log/nginx/error.log` before it reads `nginx.conf`; it is harmless when nginx runs as your user.

✓ **Checkpoint (criterion 1):** the page loads with `--cacert root_ca.pem` and `Verify return code: 0 (ok)`. nginx sends the leaf plus the intermediate (`web.crt`), so clients only need the root.

## Part D — The orders app

**Step 8 — The app.** It talks only to the Agent on 127.0.0.1:8100; it has no token and no Vault library.

```
# /opt/capstone/orders.py
"""Capstone app: card numbers are encrypted by Vault Transit through Vault Agent."""
import base64
import sqlite3
import sys

import requests

AGENT = "http://127.0.0.1:8100"      # Vault Agent API proxy - no token in this app
DB = "/opt/capstone/orders.db"


def vault(path, payload):
```

```
    r = requests.post(f"{AGENT}/v1/{path}", json=payload, timeout=5)
    r.raise_for_status()
    return r.json()["data"]


def encrypt(card):
    b64 = base64.b64encode(card.encode()).decode()
    return vault("transit/encrypt/orders", {"plaintext": b64})["ciphertext"]


def decrypt(ct):
    return base64.b64decode(vault("transit/decrypt/orders", {"ciphertext": ct})
["plaintext"]).decode()


def version(ct):
    return int(ct.split(":")[1][1:])          # "vault:v2:..." -> 2


db = sqlite3.connect(DB)
db.execute("create table if not exists orders (id integer primary key, customer text, card
text)")
cmd = sys.argv[1] if len(sys.argv) > 1 else "list"

if cmd == "add":
    cur = db.execute("insert into orders (customer, card) values (?, ?)", (sys.argv[2],
encrypt(sys.argv[3])))
    db.commit()
    print(f"order {cur.lastrowid} stored")
elif cmd == "list":
    for oid, customer, ct in db.execute("select id, customer, card from orders"):
        print(f"{oid}  {customer:<10} {decrypt(ct)}")
elif cmd == "rewrap":
    rows = db.execute("select id, card from orders").fetchall()
    latest = max(version(ct) for _, ct in rows)
    old = [(oid, ct) for oid, ct in rows if version(ct) < latest]
    for oid, ct in old:
        new = vault("transit/rewrap/orders", {"ciphertext": ct})["ciphertext"]
        db.execute("update orders set card = ? where id = ?", (new, oid))
    db.commit()
    print(f"rewrapped {len(old)} orders")
```

**Step 9 — Store, rotate, rewrap, read.** The rotation is an operator action, so it uses the root token in terminal 2, not the app.

```
$ cd /opt/capstone
$ python3 orders.py add "Asha Rao" "4111 1111 1111 1111"
$ python3 orders.py add "Ravi Iyer" "5500 0000 0000 0004"
$ sqlite3 orders.db "select id, customer, card from orders"
$ vault write -f transit/keys/orders/rotate > /dev/null
$ python3 orders.py add "Meera Das" "3400 000000 00009"
$ python3 orders.py rewrap && sqlite3 orders.db "select id, substr(card,1,9) from orders"
$ python3 orders.py list
```

Lab 3.10 · Step 9 · Orders encrypted through Agent — EXPECTED OUTPUT

```
trainee@vault-lab:~$ python3 orders.py add "Asha Rao" "4111 1111 1111 1111"
order 1 stored
trainee@vault-lab:~$ python3 orders.py add "Ravi Iyer" "5500 0000 0000 0004"
order 2 stored
trainee@vault-lab:~$ sqlite3 orders.db "select id, customer, card from orders"
1|Asha Rao|vault:v1:fV3Jt0n8m2Qq7XkY1d6bS5wL9pE4rT0uA2cZ8hG3iN1o...
2|Ravi Iyer|vault:v1:Kp0sW7yE3mA9qL2xC5vB8nR1tF6hD4jU0gZ7iO3kM2e...
trainee@vault-lab:~$ vault write -f transit/keys/orders/rotate > /dev/null
trainee@vault-lab:~$ python3 orders.py add "Meera Das" "3400 000000 00009"
order 3 stored
trainee@vault-lab:~$ python3 orders.py rewrap && sqlite3 orders.db "select id, substr(card,1,9)
                        from orders"
rewrapped 2 orders
1|vault:v2:
2|vault:v2:
3|vault:v2:
trainee@vault-lab:~$ python3 orders.py list
1  Asha Rao   4111 1111 1111 1111
2  Ravi Iyer  5500 0000 0000 0004
3  Meera Das  3400 000000 00009
```

*Expected output — only ciphertext in the database; after rotation and rewrap every row is v2; the app still reads all orders*

✓ **Checkpoint (criteria 2 and 3):** `orders.db` never contains a card number; new rows after the rotation are `vault:v2:`; after `rewrap` all rows are v2 and `list` still decrypts them.

## Part E — Renewal and evidence

**Step 10 — Agent restart reuses the certificate.** Press Ctrl+C in terminal 3 and start Agent again. Then:

```
$ grep -c '"path":"pki_int/issue/web-server"' /opt/capstone/audit.log
```

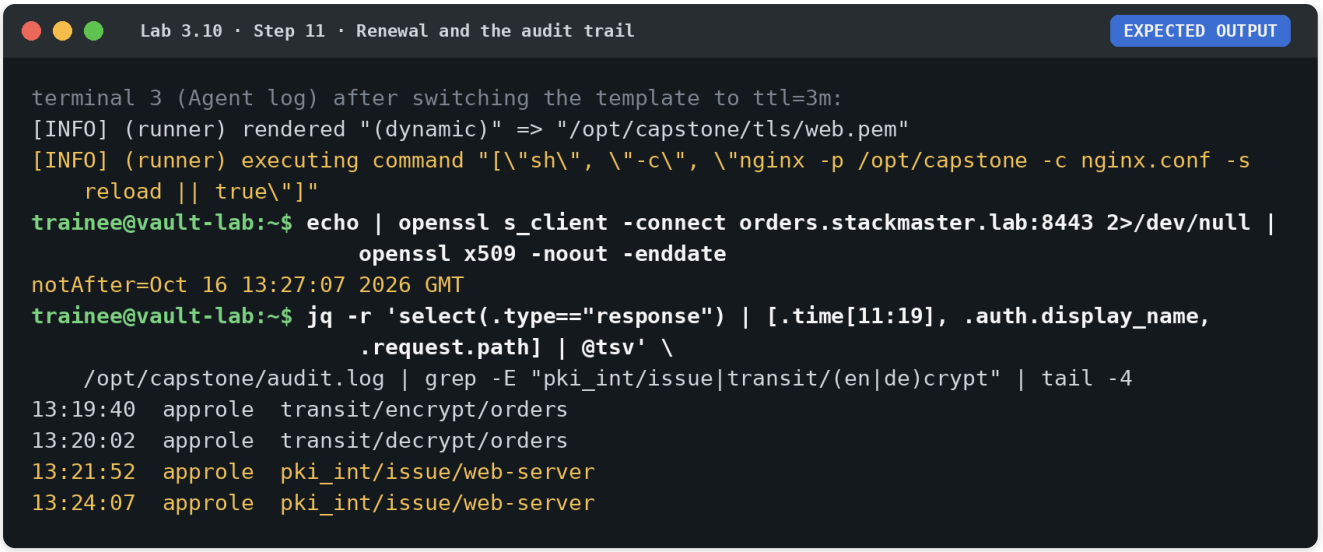
✓ **Checkpoint (criterion 4):** the count is unchanged from before the restart (each issue appears twice, request and response). `pkiCert` found a valid certificate in `web.pem` and did not issue a new one.

**Step 11 — Short-lived certificates.** Change `ttl=24h` to `ttl=3m` in `web.pem.ctmpl`, remove the cached bundle, and restart Agent:

```
$ sed -i 's/ttl=24h/ttl=3m/' /opt/capstone/web.pem.ctmpl
$ rm /opt/capstone/tls/web.pem
# terminal 3: Ctrl+C, then: vault agent -config=agent-capstone.hcl
```

Wait about three minutes, then:

```
$ echo | openssl s_client -connect orders.stackmaster.lab:8443 2>/dev/null | openssl x509 -
noout -enddate
$ jq -r 'select(.type=="response") | [.time[11:19], .auth.display_name, .request.path] |
@tsv' \
    /opt/capstone/audit.log | grep -E "pki_int/issue|transit/(en|de)crypt" | tail -4
```




*Expected output — Agent renews the 3-minute certificate and reloads nginx; the audit log shows each issue and Transit call by the approle identity*

✓ **Checkpoint (criteria 5 and 6):** Agent's log shows a new render followed by the nginx reload command; the certificate served by nginx has a new `notAfter`; the audit log lists `pki_int/issue/web-server` and `transit/...` calls by `approle` — never by a person. Set the TTL back to `24h` when you are done.

**Step 12 — Stretch goals.** See study guide 10.6: move the design to Kubernetes (Lab 3.9 patterns), express the Vault side in Terraform (Lab 3.8), or put the dev server behind Transit auto-unseal (Lab 3.3).

## Clean up

```
$ nginx -p /opt/capstone -c nginx.conf -s quit
# terminal 3: Ctrl+C (Agent); terminal 1: Ctrl+C (dev server)
$ sudo sed -i '/orders.stackmaster.lab/d' /etc/hosts
$ sudo rm -rf /opt/capstone
```

# Appendix — Reset scripts and completion checklist

Run the reset that matches where you are; each one returns the machine to a clean starting point without touching the other environments. The Day 1–2 resets are in the appendix of the Day 1–2 Lab Guide.

## Reset scripts

```
# Dev server labs (3.1, 3.2, 3.4, 3.6, 3.7): stop it and its data is gone
#   press Ctrl+C in the dev server terminal, then:
$ unset VAULT_TOKEN VAULT_CACERT; rm -f ~/.vault-token
$ export VAULT_ADDR=http://127.0.0.1:8200
# Lab 3.3 seal node
$ pkill -f "day3/seal/seal.hcl"; rm -rf ~/day3/seal
# Lab 3.4 agents (Ctrl+C in their terminals), then optionally
$ rm -rf ~/day3/agent
# Labs 3.5 and 3.8 Raft cluster
$ ~/day3/raft-cluster.sh destroy; rm -rf ~/day3/tf ~/day3/ansible
# Lab 3.6 Prometheus
$ docker rm -f prom 2>/dev/null; rm -rf ~/day3/mon
# Lab 3.7 leftovers
$ rm -rf ~/day3/breakfix ~/day3/devtls ~/day3/vault-debug.tar.gz
# Lab 3.9 cluster
$ kind delete cluster --name vault-ha
# Lab 3.10 capstone
$ nginx -p /opt/capstone -c nginx.conf -s quit 2>/dev/null
$ sudo sed -i '/orders.stackmaster.lab/d' /etc/hosts; sudo rm -rf /opt/capstone
# Everything from Day 3
$ rm -rf ~/day3
```

## Common lab problems

| Symptom | Fix |
|---|---|
| `bind: address already in use` on 8210 | The Lab 3.3 seal node is still running: `pkill -f day3/seal/seal.hcl` |
| Lab 3.4 Agent: `permission denied` on `pki_int/issue/web-server` | The dev server was restarted and lost Lab 3.1 — redo Lab 3.1 Steps 2–4 |
| Lab 3.4 Agent exits with `invalid role or secret ID` | The secret ID file was consumed: check `remove_secret_id_file_after_reading = false`, then issue a new secret ID |
| Lab 3.5: a node is missing from `list-peers` | It is still sealed or could not reach the leader: unseal it and read `~/day3/raft/<node>/vault.log` |
| Lab 3.6: Prometheus target `down` with `401` / `403` | `prom-token` not readable in the container (`chmod 644`) or the token was created on an earlier dev server |
| Lab 3.8: `path is already in use at secret/` | The `import` block is missing from `main.tf` |
| Lab 3.8: Ansible `ModuleNotFoundError: hvac` | `pipx inject ansible hvac` |
| Lab 3.9: vault-1/vault-2 stay `0/1` | They are sealed or have not joined: unseal them, and check `kubectl -n vault logs vault-1` for `retry_join` errors |

| Symptom | Fix |
|---|---|
| Lab 3.9: VSO `VaultStaticSecret` shows auth errors | Role `webapp-vso` must have `audience=vault`, matching `audiences` in the `VaultAuth` |
| Lab 3.10: curl `SSL certificate problem` | Use `--cacert /opt/capstone/tls/root_ca.pem` and the name `orders.stackmaster.lab` (check `/etc/hosts`) |
| Lab 3.10: `orders.py` gets `403` | The Agent in terminal 3 is not running, or the policy lacks the path being called |

## Completion checklist

- ☐ Lab 3.1 — root and intermediate CA built; certificate issued, verified with OpenSSL, revoked and seen on the CRL
- ☐ Lab 3.2 — card number encrypted and decrypted; key rotated; old ciphertext rewrapped; v1 refused after the floor was raised
- ☐ Lab 3.3 — seal node initialised with recovery keys and unsealed itself after a restart
- ☐ Lab 3.4 — Agent rendered KV and pkiCert templates, re-rendered on change, supervised a child process, proxied a request
- ☐ Lab 3.5 — three-node Raft cluster survived the loss of its leader; data restored from a snapshot
- ☐ Lab 3.6 — Prometheus scraped Vault; alice's read traced in the audit log; HMAC matched `audit-hash`
- ☐ Lab 3.7 — five faults fixed; `vault debug` bundle collected
- ☐ Lab 3.8 — Terraform imported `secret/` and built the rest; a policy change went through plan; Ansible took a snapshot, rotated the key and issued a certificate
- ☐ Lab 3.9 — HA Vault on kind; secret delivered by injector and VSO; rotation restarted the VSO Deployment
- ☐ Lab 3.10 — capstone meets all six success criteria from the study guide (Module 10.5)